

myoArm Forward-State-Estimation 研究遂行テキスト

理系大学院生が Project 1 を理解し、再現し、次フェーズへ拡張するために

作成日: 2026-05-12 / 対象: 修士・博士前期～博士後期初期 / Repository: myoarm-forward-state-estimation

このテキストの目的

この資料は、myoArm forward-state-estimation 研究を「読む」だけでなく、実際に再現し、結果を検証し、次の Project 2 / Project 3 に発展させるための作業教科書である。読者は Python / PyTorch / 基礎的な制御・確率の知識を持つ理系大学院生を想定する。

外部画像は使わず、リポジトリ内の生成図表と既存 primer 画像のみを用いる。

目次

1. 研究の全体像	3
1.1. 研究群の中での位置づけ	3
1.2. 中心問い	4
1.3. この資料の読み方	4
1.4. なぜ closed-loop pipeline を考えるのか	5
1.5. このループは妥当なのか	5
1.6. Downstream policy の 3 分類	6
1.7. Closed-loop pipeline を時刻順に読む	6
1.8. 1 step の擬似コード	7
1.9. delay がない場合とある場合	8
1.10. correction gain K の直感	8
1.11. logger に全層を保存する理由	8
2. 必要な前提知識	9
2.1. 筋骨格 reaching と MyoSuite	9
2.2. 信号名の区別	10
2.3. 状態ベクトル	10
2.4. 評価グリッドと cell	10
3. Forward model	11
3.1. なぜ forward model が必要か	11
3.2. 残差 MLP	11
3.3. one-step と multi-step supervision	12
4. Predictive state observer	12
4.1. 基本式	12
4.2. fixed-lag delay handling	13
4.3. 二層 reliability-adaptive observer	13
4.3.1. 二層の役割と時間スケール	13
4.3.1.1. Step-wise signal flow (within-trial)	13
4.3.1.2. Iteration-wise signal flow (across-trial)	14
4.3.1.3. 二層の関係(時系列構造)	15
4.3.1.4. なぜ二層に分けるか	16
4.3.2. Within-trial layer: innovation history から reliability へ	16
4.3.2.1. EMA(Exponential Moving Average / 指数移動平均)とは	16
4.3.2.2. Simple Moving Average との対比	17
4.3.2.3. Project 1 での具体的使用	17

4.3.2.4.	動作例((none, d=18) cell, qvel field)	17
4.3.2.5.	生物学的解釈	18
4.3.2.6.	なぜこの logistic 形で $K_f(t)$ が決まるのか	18
4.3.2.7.	β_0 と β_1 の各役割	19
4.3.2.8.	数値例: qpos field 軌道($\beta_0 = -1.5$, $\beta_1 = 0.9$ C2 学習値)	19
4.3.2.9.	Bayesian inference との対応	19
4.3.2.10.	何が決まり、何が学習されるか	20
4.3.3.	Across-trial layer: SPSA outcome adaptation	20
4.3.3.1.	なぜ最終位置でなく “最も近づいた距離” で評価するのか	21
4.3.3.1.1.	生体研究との対応	22
4.3.3.1.2.	final-tip を使うべき他の task(反例)	22
4.3.3.2.	どうやって β を更新するか — 勾配が取れない問題	22
4.3.3.3.	SPSA の詳細解説	23
4.3.3.3.1.	背景: なぜ勾配降下を直接使えないか	23
4.3.3.3.2.	Stochastic approximation の系譜	23
4.3.3.3.3.	Rademacher 摂動とは	24
4.3.3.3.4.	この (iii) がなぜ SPSA を成立させるのか	24
4.3.3.3.5.	SPSA の中心アイデア: simultaneous perturbation	25
4.3.3.3.6.	なぜこの推定が動くのか	26
4.3.3.3.7.	なぜ Rademacher で Gaussian ではないか	26
4.3.3.3.8.	Spall schedule の意味	27
4.3.3.3.9.	Paired perturbation と分散低減	27
4.3.3.3.10.	Project 1 での 1 iteration コスト	27
4.3.3.3.11.	収束の直感	28
4.3.3.3.12.	SPSA の限界と Project 1 での意味	28
4.3.4.	oracle-supervised predictor: upper bound diagnostic (Appendix C 教材)	28
5.	実験 pipeline	29
5.1.	Repository の構造	29
5.2.	最小再現手順	29
5.3.	実験 command の読み方	29
6.	主要結果の読み方	30
6.1.	4 つの主張をひとことで	30
6.2.	C1 — 初期設定の reliability ルールはタスクに合わない	30
6.2.1.	どんな実験か	30
6.2.2.	何が起きているか — 実測の per-field K_f	31
6.2.3.	なぜ届かないか	31
6.3.	C2 — 1 つの cell で SPSA を回すと、差はほぼ埋まる	31
6.3.1.	どんな実験か	31
6.3.2.	数字の読み方	32
6.3.3.	どこに「学習」が現れたか	32
6.4.	C3 — 6 cell 全部を相手にすると、改善は頭打ちになる	33
6.4.1.	どんな実験か	33
6.4.2.	数字の読み方	33
6.4.3.	なぜ頭打ちになるか — parameterisation の限界	33
6.4.4.	次のモデル形(将来研究)	34
6.5.	C4 — ルールはタスク次第:転移は必ずしも成功しない	34
6.5.1.	どんな実験か	34
6.5.2.	何が起きているか — 反転現象	34
6.5.3.	なぜ転移が失敗するか	34

7. 研究者としての作業手順	35
7.1. 1日の作業サイクル	35
7.2. 実験前チェックリスト	35
7.3. 結果解釈チェックリスト	35
8. よくある失敗と対処	36
8.1. target が paired でない	36
8.2. K と H の混同	36
8.3. behaviour-cloned policy の open-loop 化(Appendix C 教材)	36
8.4. endpoint-error policy を optimal controller と呼ぶ	36
8.5. SPSA outcome を training-time と deployed eval で混同する	36
8.6. oracle なしを “no oracle access” と書く	36
9. Project 2 / Project 3 への接続	37
9.1. Project 1.5: context-conditioned β network	37
9.2. Project 2: Bayesian integration	37
9.3. Project 3: cortico-cerebellar loop	37
10. 演習	38
10.1. 演習 1: 状態 schema を確認する	38
10.2. 演習 2: test を通す	38
10.3. 演習 3: 図表を読み解く	38
10.4. 演習 4: 新しい transfer test を設計する	38
10.5. 演習 5: context-conditioned β の prototype を設計する	38
11. 参考資料	39
11.1. Repository 内	39
11.2. Obsidian 内	39
11.3. 文献	39
12. 最後に	39

1. 研究の全体像

1.1. 研究群の中での位置づけ

本研究は myoArm 新規研究群の Project 1 である。旧 myoarm-lambda-ep の lambda-EP 単独路線から離れ、重力下の筋骨格 reaching における forward prediction、遅延感覚 feedback、不確実性統合を検証する流れの最初の本体プロジェクトである。

Project	中心問い	Project 1 との関係
Project 0	共通 myoArm 基盤	target set、logger、noise/delay wrapper、metrics を提供
Project 1	forward model + predictive state observer	本資料の対象
Project 2	Bayesian reliability-weighted integration	視覚・固有受容・prediction・prior の信頼度統合へ拡張
Project 3	cortico-cerebellar loop	residual MLP forward model を cerebellar-like module に置換
Project 4	cortical population dynamics	M1-like population dynamics へ拡張

研究ノート

Project 1 は「完成した単独テーマ」であると同時に、後続 Project の土台である。したがって、実装では再利用可能な状態表現・logger・評価 pipeline を作ることが重要になる。

1.2. 中心問い

Project 1 の中心問いは次である。

中心問い

myoArm reaching において、forward-model-based predictive state observer は、agent 自身が online で生成できるシグナル(innovation history と per-episode reaching outcome)だけから、correction gain を self-adapt できるか。

神経科学的には、これは小脳 forward model、efference copy、遅延感覚 feedback、sensory prediction error の追跡、感覚信頼度の online 推定、そして reaching outcome に基づく試行間学習を、筋骨格腕シミュレーション上で検証する第一段階である。生体は per-condition の swept oracle(K^* ラベル)を持たないので、agent-available signal だけで correction gain が決まりうるかは、forward-model framework の生物学的妥当性を問う中心的な実験となる。

工学的には、これは以下の問題になる。

前の推定状態 $\hat{x}_t$ と、前に出した運動指令 u_t

-> forward model

-> 次時刻の予測状態 $\hat{x}_{t+1}$

遅延・ノイズ付き観測 y_{t+1}

-> sensory prediction-error correction

-> 更新後の推定状態 $\hat{x}_{t+1}$

$\hat{x}_{t+1}$

-> controller

-> 次の運動指令 u_{t+1}

-> muscle excitation

-> myoArm plant

つまり、同じ $\hat{x}$ でも「更新前」と「更新後」がある。混乱を避けるため、本資料では 1 step の流れを $\hat{x}_t \rightarrow \hat{x}_{t+1} \rightarrow u_{t+1}$ と読む。

ここで $\hat{x}_{t+1}$ と $\hat{x}_t$ は役割が違う。

記号	意味	どう作るか
$\hat{x}_{t+1}$	forward model だけで作った「次はこうなっているはず」という予測状態	$\hat{x}_t$ と u_t を forward model に入れる
$\hat{x}_{t+1}$	観測も使って補正した「controller に渡す最終的な推定状態」	$\hat{x}_{t+1}$ と delayed/noisy observation y_{t+1} の差分を correction gain K で補正する

直感

$\hat{x}_{t+1}$ は「自分のモデルだけでの予想」。

$\hat{x}_t$ は「予想に、届いた感覚情報を加味して更新した現在の信念」。

controller が使うのは $\hat{x}_{t+1}$ ではなく、基本的には更新後の $\hat{x}_t$ である。

1.3. この資料の読み方

この資料は、最初から C1-C4 の論文主張を読む構成にはしていない。まず closed-loop pipeline、状態表現、forward model、predictive state observer、そして二層適応則(within-trial reliability と across-trial outcome adaptation)の意味を理解し、その後で結果 C1-C4 を読む。

推奨順序:

1. closed-loop pipeline が何のためにあるか
2. $\hat{x}_t$ / $\hat{x}_{t+1}$ / observation / action の違い

3. forward model が何を予測するか
4. sensory prediction-error correction と gain K の意味
5. innovation history からの within-trial reliability
6. SPSA across-trial outcome adaptation
7. oracle-supervised upper bound (Appendix 教材)
8. C1-C4 の結果

oracle-supervised predictor(旧 §3.4)は upper-bound diagnostic として Appendix C 化された。Project 1 の中心提案は、oracle ラベルを使わない二層適応 observer である。

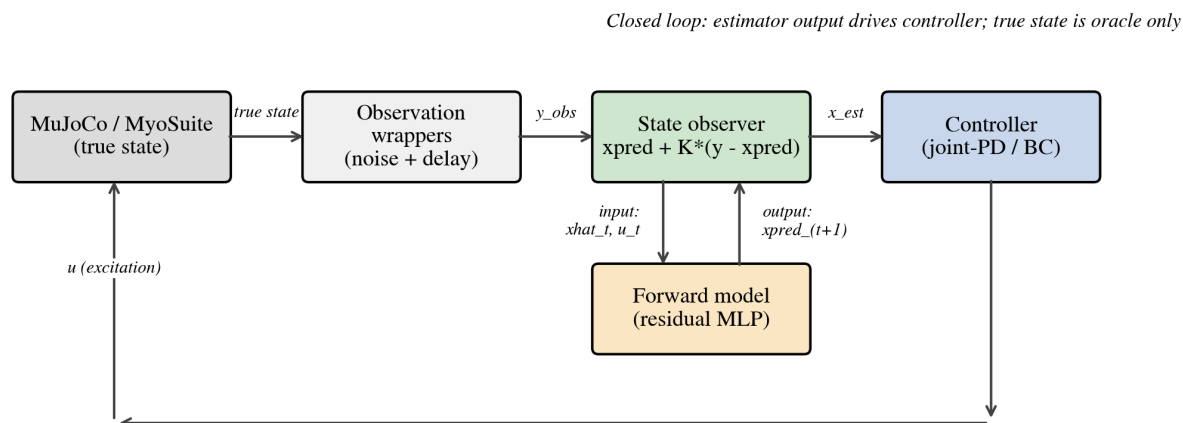


図 1: Project 1 の closed-loop pipeline。true state は評価専用で、controller は estimator output のみを見る。

1.4. なぜ closed-loop pipeline を考えるのか

この pipeline は、単にソフトウェアを複雑にしたいから作っているのではない。研究上の目的は、状態推定の質が、実際の筋骨格 reaching の制御性能に伝播するかを調べることである。

open-loop evaluation では、記録済み trajectory を replay しながら $\hat{x}_{t+1}$ と x_{t+1} の誤差を測る。これは forward model や estimator の純粋な精度を見るには有用だが、次の問いには答えられない。

closed-loop でしか答えられない問い

推定状態 $\hat{x}_{t+1}$ が少し良くなったとき、controller が実際に違う muscle excitation を出し、腕の軌道や到達誤差が変わるのか。

この問いに答えるには、estimator を controller の前段に置き、推定誤差が action を通じて plant に戻る循環を作る必要がある。これが closed-loop pipeline である。

評価	測れること	測れないこと
open-loop	state estimation error, prediction MSE, oracle K	その推定が行動に効くか
closed-loop	reaching error, success, overshoot, 推定誤差の行動への伝播	純粋な estimator 精度だけの切り分けは難しい

したがって、Project 1 では open-loop と closed-loop を両方使う。open-loop は estimator の診断、closed-loop は「その推定が制御に意味を持つか」の検証である。

1.5. このループは妥当なのか

この closed-loop は、神経科学的にも制御工学的にも妥当な抽象化である。ただし「脳を完全再現している」という意味ではない。

観点	妥当な理由	限界
----	-------	----

神経科学	efference copy に基づく forward prediction と delayed sensory feedback の統合を表す	小脳 microcircuit, spike, climbing fiber learning は再現していない
制御工学	observer + controller という標準的な閉ループ構造	gain は scalar で、完全な covariance 推定から導出した最適 gain ではない
実験設計	true state を評価専用にし、controller は estimated state だけを見るため、state estimation の効果を検証できる	controller の質が低いと estimator 差が task に出ない

Project 1 の closed-loop は、以下の仮説を検証するための最小構成である。

delayed/noisy sensory feedback
+ forward prediction
+ sensory prediction-error correction
-> better estimated current state
-> different controller action
-> different reaching trajectory

この連鎖のどこかが切れると、状態推定が良くても task performance には出ない。実際、behaviour-cloned policy では reaching は改善したが state coupling が弱く、observer signal は wash out した。逆に joint-space feedback controller と endpoint-error sensorimotor policy は state-coupled なので、推定状態の差が task 指標に現れた。

研究ノート

closed-loop pipeline の妥当性は「controller が良い」ことではなく、「controller が推定状態に依存している」ことにある。Project 1 の主結果は、状態推定の効果が state-coupled controller と observation-delay regime の組み合わせで現れる、という点である。

1.6. Downstream policy の 3 分類

論文が一段落した時点で、controller 軸は次の 3 分類に整理された。ここでは controller を広い意味で使うが、BC と endpoint-error は policy と呼ぶ方が正確である。

名前	何を見るか	位置づけ
joint-space feedback controller	joint error / velocity error から muscle excitation を出す	hand-designed, joint-level, state-coupled
endpoint-error sensorimotor policy	estimated tip-to-target error から muscle excitation を出す	task-level, endpoint-error-driven, state-coupled
behaviour-cloned policy	demonstration から state-to-action mapping を学習する	minimal learned sensorimotor policy

endpoint-error sensorimotor policy は、OFC-like controller や LQR-like controller ではない。Riccati equation、LQR、OFC を解いているわけではなく、推定された手先誤差に基づいて筋入力を補正する transparent な task-level policy である。

研究ノート

この 3 分類の意味は、「どの policy が一番 reaching できるか」だけではない。observer の出力 $\hat{x}_t$ が downstream policy の action に実際に伝わるか、つまり state coupling があるかを見るための軸である。

1.7. Closed-loop pipeline を時刻順に読む

Fig. 1 の closed-loop pipeline は、1 control step ごとに同じ処理を繰り返す。重要なのは、シミュレータ内部には常に真の状態 x_t があるが、controller はそれを直接見ないという点である。controller が見るのは、delay/noise wrapper と estimator を通った $\hat{x}_{t|t}$ だけである。

順序	モジュール	入力 -> 出力	研究上の意味
----	-------	----------	--------

1	MuJoCo / MyoSuite plant	$u_{(t-1)} \rightarrow \text{true state } x_t$	身体が実際にどう動いたか。これは評価用 oracle であり controller には渡さない。
2	state extractor	$\text{mj_data} \rightarrow \text{MyoArmState}(x_t)$	qpos, qvel, act, tip_pos, target_pos, reach_err を研究用 schema に整える。
3	observation wrappers	$x_t \rightarrow \text{delayed/noisy observation } y_t$	感覚遅延・観測ノイズを明示的に入れる。生体の sensory feedback 制約に対応。
4	forward model	$\hat{x}_{(t-1)}, u_{(t-1)} \rightarrow \text{prediction } x_{\text{pred}_t}$	efference copy と前状態推定から現在状態を予測する。
5	predictive state observer	$x_{\text{pred}_t}, y_t, \text{correction gain } K \rightarrow \text{estimate } \hat{x}_t$	sensory prediction error を補正し、controller が使う現在状態を作る。
6	controller	$\hat{x}_t \rightarrow \text{neural command / excitation } u_t$	推定状態に基づいて次の筋入力を決める。
7	ActionAdapter + SDN	$\text{excitation} \rightarrow \text{api_action}$	必要なら signal-dependent motor noise を入れ、Gym API 形式へ変換する。
8	$\text{env.step}(\text{api_action})$	$\text{api_action} \rightarrow \text{next plant state}$	次の時刻へ進む。
9	logger / metrics	true_state, observation, estimate, action	評価・学習・再現のために全層を保存する。

確認ポイント

closed-loop の核心は、true_state $\rightarrow$ observation $\rightarrow$ estimator $\rightarrow$ controller $\rightarrow$ action $\rightarrow$ plant の循環である。true_state は metrics と supervised training には使うが、closed-loop controller の入力にはしない。

1.8. 1 step の擬似コード

実装の概念は次のように読める。これは実際の関数名を完全に写したものではなく、責務を理解するための擬似コードである。

```
# t 時点。env 内部には真の MuJoCo state がある。
true_state = extract_state(env)      # oracle for logging/evaluation only

# 生体でいう sensory feedback。ここで delay/noise が入る。
obs_state = delayed_wrapper.observe(true_state)
obs_state = noisy_wrapper.observe(obs_state)

# 前 step の推定状態と実際に入れた muscle excitation から現在を予測。
x_pred = forward_model.predict(prev_estimate, prev_excitation)

# sensory prediction error を correction gain K で補正。
estimate = estimator.update(prediction=x_pred, observation=obs_state, gain=K)

# controller は estimate だけを見る。true_state は見ない。
neural_command = controller(estimate)
excitation = command_to_excitation(neural_command)
excitation = motor_noise(excitation)  # optional SDN
api_action = action_adapter.to_api(excitation)

# plant を1 step進める。
obs, reward, terminated, truncated, info = env.step(api_action)

# 後で検証できるように、true / obs / estimate / action を全部保存。
logger.append(
    true_state=true_state,
    obs_state=obs_state,
    estimate=estimate,
```



```

neural_command=neural_command,
excitation=excitation,
api_action=api_action,
)

```

このコードで最も大切なのは、`controller(estimate)` であって `controller(true_state)` ではないことだ。true state controller は oracle baseline としては有用だが、Project 1 の本筋ではない。

1.9. delay がない場合とある場合

delay が $d=0$ の場合、observation y_t は現在状態 x_t の noisy version と見なせる。このとき prediction-error correction は直感的である。

```

x_pred_t = forward_model(xhat_(t-1), u_(t-1))
y_t      = x_t + noise
xhat_t   = x_pred_t + K * (y_t - x_pred_t)

```

delay が $d>0$ の場合、 y_t は現在ではなく過去 $x_{(t-d)}$ の観測である。したがって、単純に x_{pred_t} と y_t を混ぜると時刻がずれる。Project 1 では fixed-lag buffer を使い、過去の estimate を correction してから現在まで re-roll する。

1. buffer 内に $xhat_{(t-d)}$, ..., $xhat_{(t-1)}$ と action を保持
2. y_t は $x_{(t-d)}$ の観測なので、 $xhat_{(t-d)}$ に対して correction
3. corrected $xhat_{(t-d)}$ を $u_{(t-d)}$, ..., $u_{(t-1)}$ で forward model rollout
4. 現在時刻の estimate $xhat_t$ を得る

落とし穴

delay があるときに y_t を現在状態として扱うと、推定器は「古い身体状態」を現在だと思って制御する。この誤りは reaching の振動・遅れ・overshoot として現れる。

1.10. correction gain κ の直感

ここでの κ は Kalman filter から導出した gain ではない。 κ は sensory prediction error に対する correction gain であり、forward prediction と sensory feedback のどちらをどの程度信じるかを表す実験上の knob である。

K	推定器の挙動	closed-loop で起きやすいこと
K=0	prediction-only。observation correction を使わない。	forward model が強ければ delay を回避できるが、model error があると発散する。
K=1	observation-only。prediction は correction 後の re-roll 以外では弱い。	観測が新鮮なら強い。delay が大きいと古い情報に引っ張られる。
$0<\kappa<1$	prediction と observation の blending。	観測ノイズが大きく、delay が小さい条件では有利になりやすい。

この研究で重要だったのは、open-loop state estimation error を最小にする κ と、closed-loop task error を最小にする κ が一致しない場合があることだった。つまり「状態推定として正確」でも「制御に使うと良い」とは限らない。

1.11. logger に全層を保存する理由

closed-loop pipeline では、1 step ごとに似たような vector が複数出てくる。これらを保存しないと、失敗したときに原因を切り分けられない。

保存する量	用途	典型的な診断
true_*	oracle evaluation / supervised training	estimation error が本当に下がったか
obs_*	delay/noise 後の観測	wrapper の設定が効いているか
estimate_*	controller 入力	controller が何を信じて動いたか
neural_command	抽象的 controller output	将来の cortical module との接続
excitation	canonical muscle input	SDN 後に何が plant に入ったか

api_action	Gym API input	ActionAdapter の変換確認
activation	muscle internal state	筋 dynamics の遅れや飽和

演習 / 作業

closed-loop の結果を読むときは、まず `final_tip_error` を見るのではなく、同じ episode の `true_tip_pos`, `obs_tip_pos`, `estimated_tip_pos`, `excitation` を並べる。どの層で差が生まれたかを見ないと、estimator の効果と controller の癖を混同する。

2. 必要な前提知識

2.1. 筋骨格 reaching と MyoSuite

MyoSuite myoArm は、MuJoCo 上で動く筋骨格腕モデルである。Project 1 で主に扱うのは、示指先端を target に近づける reaching task である。

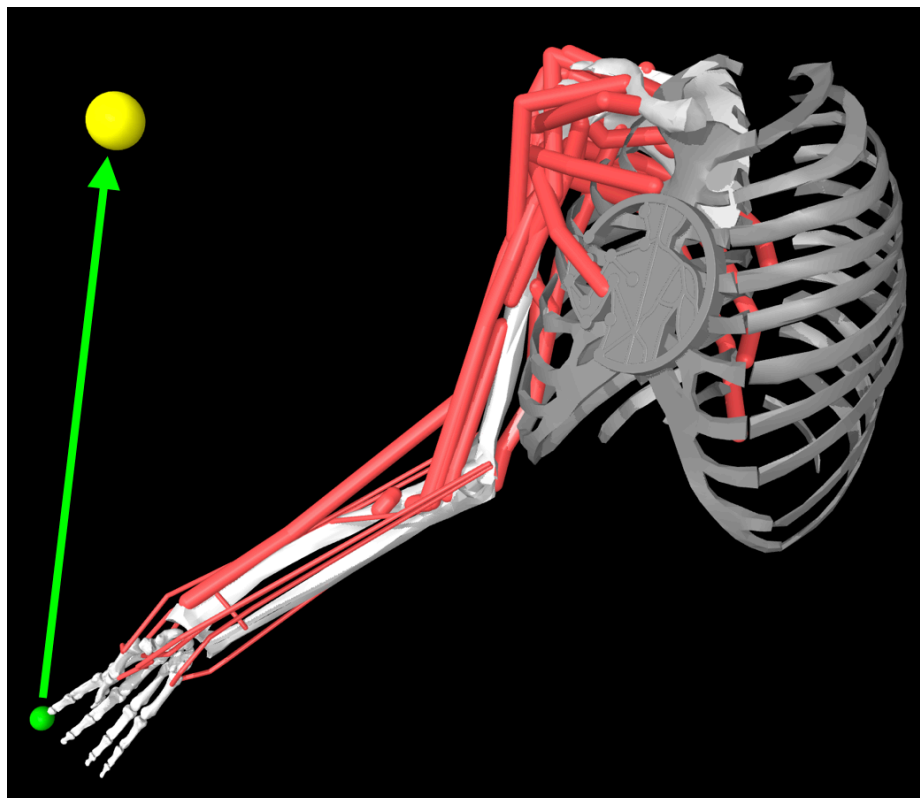


図 2: myoArm reaching task の概念図。リポジトリ内 primer の既存画像を使用。

落とし穴

この図の胸部付近にある円形マークは、元画像に含まれるロゴ / 透かしであり、筋・骨・関節・センサーなどのモデル要素ではない。著作権表示や出典表示を意図したマークを削除・改変して使ってはいけない。投稿・配布用の図では、(1) 元画像を改変せず出典とライセンスに従って使う、または (2) 自分で MuJoCo からレンダリングした図に差し替える、のどちらかにする。

項目	値 / 形	注意
muscles	34	action / excitation dimension
joint position	$\mathbf{q}$ in $\mathbb{R}^{20}$	MyoSuite / MuJoCo の <code>qpos</code>
joint velocity	$\dot{\mathbf{q}}$ in $\mathbb{R}^{20}$	<code>qvel</code>
activation	$\mathbf{a}$ in $\mathbb{R}^{34}$	筋内部状態。command ではない
control step	0.02 s	MuJoCo internal timestep 0.002 s と混同しない

episode	600 steps = 12 s	horizon と max_steps を揃える
---------	------------------	--------------------------

落とし穴

MyoSuite の observation は便利だが、研究用の状態 schema と一致するとは限らない。本研究では mj_data から true state を抽出し、MyoArmState として明示的に管理する。

2.2. 信号名の区別

この研究で最も重要な実装原則は、作用信号の層を混同しないことである。

neural_command controller が出す抽象的な運動指令。将来、cortical population dynamics の出力になる可能性がある。	excitation 研究側 canonical 表現。筋 actuator 入力として解釈する $[0,1]^{34}$ 。
api_action Gym / MyoSuite の env.step() に渡す $[-1,1]^{34}$ 。	activation 筋モデル内部の活性化状態。controller output ではない。
mj_data.ctrl MuJoCo の actuator control。post-step inspection 用。	true_state シミュレータ内部の真値。oracle baseline と評価以外では controller に渡さない。

基本変換は次である。

```
api_action = 2.0 * np.clip(excitation, 0.0, 1.0) - 1.0
excitation = (np.clip(api_action, -1.0, 1.0) + 1.0) / 2.0
```

確認ポイント

論文・コード・ログで action とだけ書かれていたら、そのたびに「どの層の action か」を確認する。特に SDN は api_action ではなく neural_command / excitation 側に入れる。

2.3. 状態ベクトル

Project 1 の状態は次の 83 次元ベクトルである。

```
x_t = [qpos, qvel, act, tip_pos, target_pos, reach_err]
dim = 20 + 20 + 34 + 3 + 3 + 3 = 83
```

ここで reach_err は、現在の論文では target_pos - tip_pos として扱う。

field	dim	意味
qpos	20	関節位置
qvel	20	関節速度
act	34	筋 activation
tip_pos	3	示指先端位置
target_pos	3	target 位置
reach_err	3	target_pos - tip_pos

2.4. 評価グリッドと cell

Project 1 で頻出する cell という用語の意味を先に定義しておく。

定義: cell

cell = 評価グリッドの 1 区画。具体的には (noise level, delay) の組合せ 1 つを指す。

observer の性能は、観測ノイズの強さと sensor 遅延の組合せで挙動が大きく変わる。Project 1 はこれを系統的に測るため、focused grid と呼ばれる以下の 6 区画でほぼすべての closed-loop 評価を行う。

noise level	delay	cell ラベル	挙動の傾向
none(0)	0 step	(none, d=0)	理想に近い: sensor 即時 + ノイズなし
none	18 step	(none, d=18)	delay 主導: forward model rollout が効く
high(中)	0	(high, d=0)	noise 主導: 観測が荒れる
high	18	(high, d=18)	noise + delay の複合
xhigh(大)	0	(xhigh, d=0)	強 noise: sensor 強く不信
xhigh	18	(xhigh, d=18)	強 noise + 大 delay の最難条件

これら 6 つを 3 noise $\times$ 2 delay = 6 cell と数える。論文中の “delay-18 cells” は (none, d=18) / (high, d=18) / (xhigh, d=18) の 3 cell、“delay-0 cells” は対応する d=0 の 3 cell を指す。

focused grid

Project 1 main results が走る 3 noise $\times$ 2 delay = 6 cell の評価盤。各 cell につき 10 episode 走らせ、10 episode の平均 min-tip を per-cell の代表値とする。論文の全 figure / table はこの 6 cell の上で読む。

full stress grid

Appendix C(oracle-supervised diagnostic)が使うより広い評価盤 = 6 noise $\times$ 4 delay $\times$ 3 controller = 72 cell。oracle K_{ol}^* を per-cell に sweep するために必要。main results では使わない。

なぜ cell ごとに分けるか: 同じ observer でも noise / delay の組合せが違えば innovation の振る舞いが変わり、closed-loop oracle K_{cl}^* も変わりうる。cell ごとに評価することで、どの条件で improving し、どの条件で破綻するかを切り分けられる。C1 / C2 / C3 / C4 の主張はすべて「どの cell でどう振る舞うか」を per-cell で示している。

3. Forward model

3.1. なぜ forward model が必要か

感覚 feedback に delay がある場合、時刻 t に届く observation は過去の状態 x_{t-d} に対応する。したがって controller が現在状態 x_t に基づいて動くには、過去の観測を forward model で現在まで転がす必要がある。

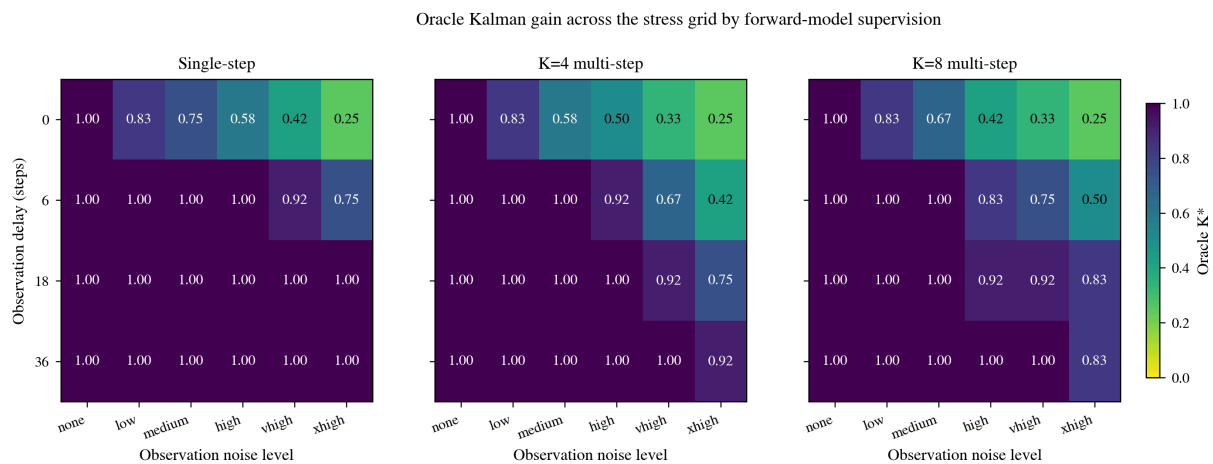


図 3: open-loop oracle gain の heatmap。forward model の訓練方法により、どの delay/noise 条件で prediction を信じるべきかが変化する。

3.2. 残差 MLP

Project 1 の baseline forward model は residual MLP である。

input: $[x_t, u_t]$ in $\mathbb{R}^{(83 + 34)}$

output: Δx_t in $\mathbb{R}^{83}$

predict: $\hat{x}_{t+1} = x_t + f_{\theta}(x_t, u_t)$

残差形式を使う理由は、control step が短く、隣接状態の差分が小さいためである。モデルは identity map を再学習するのではなく、状態変化だけを学ぶ。

3.3. one-step と multi-step supervision

one-step loss:

$$L_1(\theta) = \mathbb{E}[\|f_{\theta}(x_t, u_t) - (x_{t+1} - x_t)\|^2]$$

multi-step rollout loss:

$$L_{H(\theta)} = \mathbb{E}\left[\frac{1}{H} \sum_{h=1}^H \|\hat{x}_{t+h} - x_{t+h}\|^2\right]$$

ここで H は rollout supervision horizon である。論文の中では、correction gain の K と混同しないよう、rollout horizon を H と呼ぶ。

訓練	利点	リスク
$H=1$	実装が簡単、one-step MSE が下がりやすい	long rollout で誤差が蓄積
$H=4$	短期 rollout を安定化	訓練が重い
$H=8$	closed-loop で prediction-only が使える領域が広がる	open-loop oracle と closed-loop optimum の乖離が大きくなる

研究ノート

Project 1 の重要な発見は、forward model を強くすると「よりよい blending」が得られるだけでなく、closed-loop objective では $K=0$ prediction-only が支配的になる条件が現れることだった。

4. Predictive state observer

4.1. 基本式

Project 1 の推定器は、完全な Kalman filter ではない。神経科学的に重要なのは Kalman 最適性ではなく、efference copy に基づく forward prediction と、遅延・ノイズ付き sensory feedback から生じる sensory prediction error を、closed-loop 行動に有用な形で統合できるかである。

最小形の update は次である。

$$\hat{x} = x_{\text{pred}} + K(y - x_{\text{pred}})$$

ここでは K は $[0,1]$ の scalar として全 field に broadcast する。 K は Kalman covariance から計算した gain ではなく、sensory prediction error correction gain である。

落とし穴

この推定器を「Kalman filter」と呼ばない。共分散 P 、process noise Q 、observation noise covariance R を伝播して最適 gain を計算しているわけではない。本文では predictive state observer または forward-model-based observer と呼び、Kalman は「innovation update と似た形を持つ」という補足に留める。

この式の各項は次のように読む。

項	意味	注意
x_{pred}	forward model だけで作った予測	まだ観測で補正していない
y	遅延・ノイズ付き観測	delay がある場合は現在ではなく過去の状態を見ている
$y - x_{\text{pred}}$	sensory prediction error、予測と観測のずれ	神経科学的には感覚誤差補正信号として読む
K	prediction error をどれだけ推定に反映するか	0 なら感覚誤差を無視、1 なら観測に寄せる
$\hat{x}$	補正後の推定状態	controller に渡す状態

したがって x_{pred} は estimator の途中結果であり、 $\hat{x}$ / x_{hat} は estimator の出力である。

K	意味	直感
0	prediction-only	forward model を完全に信じる
1	observation-only	delayed/noisy observation を完全に信じる
$0 < K < 1$	blending	prediction と observation を混ぜる

4.2. fixed-lag delay handling

delay が d step ある場合、観測 y_t は現在ではなく $t-d$ の状態に対応する。そこで estimator は過去の estimate と action buffer を保持する。

buffer: $\hat{x}_{t-d}, \dots, \hat{x}_{t-1}$

obs: $y_t \sim x_{t-d} + \text{noise}$

1. delayed buffer entry $\hat{x}_{t-d}$ を observation で correction
2. corrected $\hat{x}_{t-d}$ を action buffer で t まで re-roll
3. controller は present estimate $\hat{x}_t$ を使う

落とし穴

delay wrapper の意味を変えない。Project 1 では $\text{observe}(s_t) \rightarrow s_{t-d}$ とし、delay buffer length と correction semantics を一貫させる。

4.3. 二層 reliability-adaptive observer

ここまでの定式化では K は固定 scalar である。Project 1 の中心提案は、 K を agent 自身が online で生成できる信号 から動的に決める二層適応則である。

二層適応則の骨格

1. Within-trial layer: 各 sensory field の innovation 二乗を EMA で追跡 $\rightarrow$ reliability $r_{f(t)} \rightarrow$ logistic で per-field gain $K_{f(t)}$ 。
2. Across-trial layer: SPSA で meta-parameter $\beta = \{\beta_{0,f}, \beta_{1,f}\}$ を per-episode reaching outcome から更新。

agent-available signals

agent 自身が観測・計算可能なシグナルのこと。 y_t (観測)、 $\hat{x}_{t-d}$ (自分の推定)、 u_t (自分の出した運動指令)、reach 終了後の $\min_t |tip - target|$ などが含まれる。 x_t (真の状態)や per-condition の K -sweep oracle ラベルは含まれない。

4.3.1. 二層の役割と時間スケール

ここでいう「trial」は 1 回の reach episode(600 step $\times$ 0.02 s = 12 秒)を指す。二層適応則は、この trial の中 と 外 で異なる学習を回す。

	within-trial layer	across-trial layer
時定数	~ 20 step (≈ 0.4 s)	~ 100 trial(数十分～数時間)
信号源	step ごとの innovation $e_t = y_t - \hat{x}_{t-d}$	trial 終了後の outcome $\min \text{Tip}(\beta)$
何が変わるか(state)	field-wise EMA variance $v_{f(t)}$ 、つまり gain $K_{f(t)}$ そのもの	meta-parameter $\beta = \{\beta_{0,f}, \beta_{1,f}\} \in \mathbb{R}^{10}$
何が固定(input)	β は固定 const として与えられる	β を動かす(within-trial dynamics ごと書き換える)
生物学的読み(coarse)	小脳 microcircuit で常時走る per-channel error monitoring	シナプス可塑性 / consolidation レベルの slow learning

4.3.1.1. Step-wise signal flow (within-trial)

within-trial layer は 1 episode 内で step $t = 1, 2, \dots, T$ ($= 600$) について繰り返す手続きである。各 step での処理は次の 7 段。

step	操作	式 / 説明
①	observation 到着	$y_t = h(x_{t-d}) + \eta_t$ (h は identity / y_t と x_{t-d} は同じ schema)
②	innovation 計算	$e_t = y_t - \hat{x}_{t-d}$ ($\hat{x}_{t-d}$ は buffer に保存してある $t-d$ 時点の自分の推定)
③	field-wise EMA 更新	$v_{f(t)} = (1 - \alpha)v_{f(t-1)} + \alpha \cdot \frac{1}{ \mathcal{J}_f } \sum_{i \in \mathcal{J}_f} e_{t,i}^2$ (5 field 並列、各 field の指数 i にわたって 2 乗平均)
④	reliability 化	$r_{f(t)} = \frac{1}{\varepsilon + v_{f(t)}}$ (v_f 小 $\rightarrow r_f$ 大 $\rightarrow$ sensor 信頼できる)
⑤	logistic gain	$K_{f(t)} = \sigma(\beta_{0,f} + \beta_{1,f} \log r_{f(t)})$ (β は across-trial layer が与える固定 const)
⑥	buffer 上の補正	$\hat{x}_{t-d} \leftarrow \hat{x}_{t-d} + K(t) \odot (y_t - \hat{x}_{t-d})$ ($K(t) \in [0, 1]^{83}$ は $K_{f(t)}$ を field 内で broadcast したベクトル、 $\odot$ は要素積)
⑦	re-roll 前進	補正済み $\hat{x}_{t-d}$ を action buffer の $u_{t-d}, u_{t-d+1}, \dots, u_{t-1}$ を順次適用して forward model で前進、現在時刻の estimate $\hat{x}_t$ を得る

各記号の型と意味:

記号	型	意味
t	$\text{int} \in [1, T]$	step index($T = 600$, $\Delta t = 0.02$ s)
d	$\text{int} \geq 0$	observation delay step 数(0 または 18)
x_t	$\mathbb{R}^{83}$	真の状態(評価専用、observer は見ない)
y_t	$\mathbb{R}^{83}$	delay + noise を経た観測ベクトル
η_t	$\mathbb{R}^{83}$	field ごとの i.i.d. Gaussian 観測ノイズ
$\hat{x}_t$	$\mathbb{R}^{83}$	observer の出す現在 estimate
$\hat{x}_{t-d}$	$\mathbb{R}^{83}$	buffer 内に保存された $t-d$ 時点の estimate
e_t	$\mathbb{R}^{83}$	innovation vector(observation - delayed estimate)
$\mathcal{J}_f$	set of int	field f が占める state vector のインデックス集合
$ \mathcal{J}_f $	int	field f の次元(qpos:20 / qvel:20 / act:34 / tip_pos:3 / reach_err:3)
$v_{f(t)}$	$\mathbb{R}_{\geq 0}$	field f の EMA innovation 2 乗平均(per-field スカラ)
α	$(0, 1)$	EMA 学習率(0.05、effective window ~ 20 step)
$v_{f(0)}$	$\mathbb{R}_{> 0}$	EMA 初期値(1.0、 $K_{f(0)} = \sigma(\beta_{0,f})$ になる)
$r_{f(t)}$	$\mathbb{R}_{> 0}$	field f の reliability(per-field スカラ)
ε	$\mathbb{R}_{> 0}$	数値安定化定数(10^{-6})
$\beta_{0,f}$	$\mathbb{R}$	field f の logistic intercept($r_f = 1$ 時の baseline gain $\sigma(\beta_{0,f})$)
$\beta_{1,f}$	$\mathbb{R}$	field f の logistic slope(reliability への感応度)
$\sigma(z)$	$(0, 1)$	logistic sigmoid $\frac{1}{1+e^{-z}}$
$K_{f(t)}$	$[0, 1]$	field f の correction gain(per-field スカラ)
$K(t)$	$[0, 1]^{83}$	83 次元 gain vector($K_{f(t)}$ を $\mathcal{J}_f$ 上に broadcast)
u_t	$\mathbb{R}^{34}$	muscle excitation コマンド
f_θ	forward model	residual MLP $\hat{x}_{t+1} = \hat{x}_t + f_\theta(\hat{x}_t, u_t)$

ここで学習されているのは $v_{f(t)}$ という EMA state(memory cell に近い per-field スカラ動的状態)である。重みパラメータ β は within-trial layer の内側では固定。episode 開始時に $v_{f(0)} = 1.0$ から始まり、その episode 中だけ shape される。episode が終わると v_f もリセットされる(動態は trial に閉じている)。

4.3.1.2. Iteration-wise signal flow (across-trial)

across-trial layer は episode を beach iteration の単位として SPSSA で β を更新する手続き。Spall (1992) の Simultaneous Perturbation Stochastic Approximation を Project 1 の設定に書き下したもの。

step	操作	式 / 説明
------	----	--------

①	step size 計算	$a_n = a/(n+1+A)^{\alpha_s}$ $c_n = c/(n+1)^{\gamma_s}$ $(a = 2.0, c = 0.3, A = 5, \alpha_s = 0.602, \gamma_s = 0.101, \text{Spall 推奨})$
②	Rademacher 摂動	$\Delta_n \in \{-1, +1\}^{10}$ を一様独立にサンプル(10 次元 β ごと ± 1)
③	$\pm$ 側で S 個 episode 評価	$\beta_n^+ = \beta_n + c_n \Delta_n, \beta_n^- = \beta_n - c_n \Delta_n$ 各 β で S 個 episode を独立 seed で走らせ、minTip 平均を取る: $o_n^+ = \frac{1}{S} \sum_{s=1}^S \text{minTip}(\beta_n^+, \text{seed}_{n,s,+})$ $o_n^- = \frac{1}{S} \sum_{s=1}^S \text{minTip}(\beta_n^-, \text{seed}_{n,s,-})$
④	SPSA 勾配推定	$\hat{g}_n = \frac{o_n^+ - o_n^-}{2c_n} \cdot \Delta_n^{-1}$ $(\Delta_n^{-1}$ は要素単位の数値の逆数 = Rademacher なので Δ_n 自身に等しい)
⑤	β 更新	$\beta_{n+1} = \beta_n - a_n \hat{g}_n$ $(\beta \text{ の全 } 10 \text{ 次元を } 1 \text{ 度の摂動 } \Delta_n \text{ で同時更新})$

各記号の型と意味:

記号	型	意味
n	$\text{int} \in [0, N-1]$	SPSA iteration index($N = 100$)
β_n	$\mathbb{R}^{10}$	iter n 時点の meta-parameter $= (\beta_{0,\text{qpos}}, \beta_{0,\text{qvel}}, \beta_{0,\text{act}}, \beta_{0,\text{tip_pos}}, \beta_{0,\text{reach_err}},$ $\beta_{1,\text{qpos}}, \beta_{1,\text{qvel}}, \beta_{1,\text{act}}, \beta_{1,\text{tip_pos}}, \beta_{1,\text{reach_err}})$
β_0 (初期値)	$\mathbb{R}^{10}$	$\{\beta_{0,f=0}, \beta_{1,f=0.5}\}$ (default reliability prior)
Δ_n	$\{-1, +1\}^{10}$	Rademacher perturbation vector(1 iter ごと独立)
a_n, c_n	$\mathbb{R}_{>0}$	Spall step size / perturbation amplitude
a, c, A	$\mathbb{R}_{>0}$	Spall 定数(2.0, 0.3, 5)
α_s, γ_s	$(0, 1)$	Spall 指数(0.602, 0.101, 収束のための漸近条件を満たす)
S	int	paired samples per side(single-cell:10 / full-grid:12)
β_n^+, β_n^-	$\mathbb{R}^{10}$	perturbed evaluation 点
o_n^+, o_n^-	$\mathbb{R}_{\geq 0}$	$\pm$ 側の minTip 平均(meters)
$\text{minTip}(\beta, \text{seed})$	$\mathbb{R}$	1 episode の min-tip distance(scalar outcome)
$\hat{g}_n$	$\mathbb{R}^{10}$	SPSA 勾配推定
N	int	iteration 総数(100)

ここで学習されているのは β そのもの。within-trial layer の動態 v_f は触らない。SPSA は 10 次元の β 空間を 1 iteration あたり 2 評価(forward と backward の paired perturbation)で探索する有限差分なら $2 \times 10 = 20$ 評価必要なところを 2 評価に圧縮する。Rademacher 摂動の使用と Spall schedule の組合せで確率的に局所最小に収束することが理論的に保証されている。

4.3.1.3. 二層の関係(時系列構造)

within-trial layer と across-trial layer の関係を時系列に並べると次のようになる。 β は across-trial が更新し、その β を入力として within-trial が 1 episode 走る、というネスト構造。

	trial n	trial $n+1$	trial $n+2$
入力 β	β_n (across-trial が直前に更新)	β_{n+1}	β_{n+2}
within-trial 動態	$v_{f(0)} = 1.0 \rightarrow v_{f(T)}$ を T step 進化	同左	同左
step-wise gain	$K_{f(0)}, K_{f(1)}, \dots, K_{f(T)}$ を生成	同左	同左
trial 出力	minTip_n (scalar)	minTip_{n+1}	minTip_{n+2}

表 1: within-trial 動態は trial ごとに $v_{f(0)} = 1$ にリセットされ、 β_n を const として $T = 600$ step 進化する。最終的に scalar outcome minTip_n を出力する。

across-trial layer は、複数の trial outcome を集めて β を更新する:

trials within one SPSA iteration n:

```
draw Delta_n ~ Rademacher({-1, +1})^10
collect S paired episodes for beta_n + c_n Delta_n -> o_n^+
collect S paired episodes for beta_n - c_n Delta_n -> o_n^-
compute g_hat_n = (o_n^+ - o_n^-) / (2 c_n) * Delta_n^{-1}
update beta_{n+1} = beta_n - a_n * g_hat_n
```

つまり:

- within-trial layer は β を入力 const に取り、innovation history を進化させる
- across-trial layer は trial 列の outcome 集合を入力に取り、 β を書き換える
- 次の trial set からは新しい β で within-trial 動態が走る

二層の責任分担

within-trial: "今この瞬間 sensor をどれくらい信じるか" を innovation の流れから決める(短時間スケールの reliability tracking)。

across-trial: "そもそも reliability から gain への写像をどう設計するか" を outcome から決める(長時間スケールの meta-learning)。

4.3.1.4. なぜ二層に分けるか

C1(default reliability が task-misaligned)が直接の動機。

1. innovation を見れば sensor 信頼度 は分かる。だが「forward model が task に十分良いから sensor は要らない」は分からない。これは 試行の結末(= reach の outcome)を見ないと判別できない。
2. 一方で、毎 step outcome 信号を待つことはできない。reach が終わるまで minTip は確定しない。
3. したがって innovation で step ごとに即応(within-trial)+ outcome で trial 間で slow correction(across-trial)という二層構成が自然な解になる。

C2 の主結果 = across-trial layer が 1 cell で 17 / 23 cm の gap を埋める = 「outcome を加えて初めて task-aware adaptation になる」を直接示している。C3 の限界 = β 一つで multi-cell に対応できない = across-trial layer の parameterisation 限界(SPSA 限界ではない)。次の自然な拡張は β 自体を context-conditioned network 化すること(Project 1.5)。

4.3.2. Within-trial layer: innovation history から reliability へ

各 step、観測 y_t と過去 estimate の差分(innovation)を計算する。

$$e_t = y_t - \hat{x}_{t-d}$$

これは古典的 Kalman の innovation と同じ residual である。Project 1 ではこの 83 次元 vector を 5 つの sensory field に分けて扱う。

field	次元	生物学的読み(coarse)
qpos	20	proprioceptive(joint position)
qvel	20	proprioceptive(joint velocity)
act	34	efferent / muscle-command-like
tip_pos	3	visual / task-space
reach_err	3	visual / task-space(target との差)

target_pos(3 次元)は信頼度学習の対象外。target は既知量で prediction error が意味を持たない。

4.3.2.1. EMA(Exponential Moving Average / 指数移動平均)とは

時系列 $x_1, x_2, \dots, x_t$ の「現在の平均」を、過去ほど指数的に重みを下げて推定する手法。再帰的に次で定義する。

$$v(t) = (1 - \alpha) \cdot v(t-1) + \alpha \cdot x_t$$

- $\alpha \in (0, 1)$: smoothing factor / 学習率
- $v(0)$: 初期値(Project 1 では $v_f(0) = 1$)

この再帰を展開すると、

$$v(t) = \alpha x_t + \alpha(1 - \alpha)x_{t-1} + \alpha(1 - \alpha)^2 x_{t-2} + \dots$$

過去のデータには $(1 - \alpha)^k$ で指数減衰する重みがかかる。重みの大半が含まれる effective window は近似的に $\frac{1}{\alpha}$ step。

α	effective window	性格
0.5	~ 2 step	ほぼ直近の値そのもの
0.1	~ 10 step	短期平均
0.05 (Project 1)	~ 20 step	paper の中時間スケール(≈ 0.4 s)
0.01	~ 100 step	長期 trend
0.001	~ 1000 step	ほぼ episode 全体

Project 1 の $\alpha = 0.05$ は、step = 0.02 s で ≈ 0.4 秒の窓。reach episode 12 秒の 3% 程度。瞬間 noise spike には過剰反応せず、reach 中盤での sensor 信頼度変化には追従できる選び方。

4.3.2.2. Simple Moving Average との対比

	SMA(window N)	EMA(rate α)
定義	$v(t) = \frac{x_{t-N+1} + \dots + x_t}{N}$	$v(t) = (1 - \alpha)v(t - 1) + \alpha x_t$
memory	window N 個の過去値を保持	スカラ $v(t - 1)$ 1 個だけ
計算量	$O(N)$ per step	$O(1)$ per step
窓の縁	hard cutoff($N + 1$ step 前は重み 0)	soft cutoff(指数減衰)
直近重視	全要素均等	直近に重み大

EMA の利点は memory が 1 スカラで済むこと。Project 1 では 5 つの sensory field $\times$ per-step 更新なので、SMA だと window 長さ分の buffer $\times 5$ field 必要。EMA なら $v_{f(t)}$ を 5 個のスカラで完結できる。

4.3.2.3. Project 1 での具体的使用

各 field の 2 乗 innovation 平均を EMA で追跡する。

$$v_{f(t)} = (1 - \alpha)v_{f(t-1)} + \alpha \cdot \text{mean}_{i \in \mathcal{I}_f}(e_{t,i}^2)$$

alpha=0.05 だと時定数 20 step、1 episode の 3% 程度の窓。v_f が小さい = innovation が一貫して小さい = sensor 信頼できる。逆に大きい = sensor 信頼できない。

なぜ二乗を取るか

innovation e_t そのものの平均は、forward model が unbiased なら(平均的に) 0 に近い。「sensor が信頼できるか」を測りたいので、分散 / 二乗誤差 を見たい。二乗平均は分散の推定量(mean が 0 と仮定すれば $E[e^2] = \text{Var}(e)$)。これは古典的 adaptive Kalman filter(Mehra 1970)が innovation sequence から noise covariance を推定するのと同じ発想。

4.3.2.4. 動作例((none, d=18) cell, qvel field)

t=0: v_qvel = 1.0 (初期値)
t=1: e_t^2 = 0.02 (sensor 信頼できる)
v_qvel = 0.95·1.0 + 0.05·0.02 = 0.951
t=2: e_t^2 = 0.05
v_qvel = 0.95·0.951 + 0.05·0.05 = 0.906
...
t=200: v_qvel $\approx$ 0.4 (running variance に近づく)
t=400: v_qvel $\approx$ 0.38 (定常状態)

v_qvel が下がる $\rightarrow$ reliability $r_f = \frac{1}{\varepsilon + v_f}$ が上がる $\rightarrow$ logistic K_f が大きくなる $\rightarrow$ sensor を信じる。逆に innovation が暴れる cell では v_f が高止まりし、 K_f は下がる。

Project 1 の tab:default_kf で qvel だけが $K = 0.31 - 0.49$ と他 field より低い水準に張り付くのは、qvel の innovation 二乗が比較的高め(joint velocity は MyoSuite では振動成分が乗りやすい)で reliability が抑えられているため。

4.3.2.5. 生物学的解釈

EMA は神経生物学的にも自然な実装。脳の neural integrator(leaky integrator)は連続時間で

$$\tau \dot{v}(t) = -v(t) + \text{input}(t)$$

を解く回路。これを Δt で離散化すると

$$v(t + \Delta t) = \left(1 - \frac{\Delta t}{\tau}\right)v(t) + \left(\frac{\Delta t}{\tau}\right)\text{input}(t)$$

これは EMA 形式と等価で、 $\alpha = \frac{\Delta t}{\tau}$ と同定できる。つまり EMA の α は神経 leaky integrator の time constant τ の逆数に対応する。

biological reading

Project 1 の within-trial layer は、innovation の二乗を入力とする生物学的 leaky integrator と読める。小脳 / 大脳基底核の short-time-scale neural integrator が、step ごとの sensory prediction error の大きさを統合し続けている、というモデルに対応する。

reliability に変換:

$$r_{f(t)} = \frac{1}{\varepsilon + v_{f(t)}}$$

そして logistic で per-field gain $K_{f(t)}$ に写像:

$$K_{f(t)} = \sigma(\beta_{0,f} + \beta_{1,f} \log r_{f(t)})$$

parameter	意味	直感
$\beta_{0,f}$	intercept	$r_f = 1(\log r = 0)$ 時の baseline gain $K_f = \sigma(\beta_{0,f})$ 。例えば $\beta_{0,f} = -1.5$ なら baseline $K_f \approx 0.18$ 。
$\beta_{1,f}$	slope	reliability の変化に gain がどれだけ強く反応するか。 $\beta_{1,f} = 0$ なら gain は固定、 $\beta_{1,f}$ が大きいほど innovation 増減で gain が大きく動く。

直感

innovation が暴れる → reliability が下がる → K_f が小さくなる → sensor を信じず forward model を信じる。

innovation が落ち着く → reliability が上がる → K_f が大きくなる → sensor を信じる。

この per-field 動的調節を innovation history だけ から行うのが within-trial layer。

4.3.2.6. なぜこの logistic 形で $K_{f(t)}$ が決まるのか

$K_{f(t)} = \sigma(\beta_{0,f} + \beta_{1,f} \log r_{f(t)})$ は 3 つの要請を同時に満たす最小の関数形である。それぞれ独立に動機がある。

要請 1: $K_f \in [0, 1]$ を保証したい

innovation correction は $K \cdot e_t$ を $\hat{x}$ に足す形なので、 K が範囲外に出ると blending が壊れる。任意の入力に対して必ず $(0, 1)$ に収まる滑らかな関数 → ロジスティック sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ 。

要請 2: 入力 r_f は $(0, \infty)$ なので変換が必要

$r_f = \frac{1}{\varepsilon + v_f}$ は variance の逆数なので正のスカラー。 σ の入力は $\mathbb{R}$ なのでマッピングが必要。**log** を取る: $\log r_f : (0, \infty) \rightarrow \mathbb{R}$ で σ にきれいに渡せる。

- $r_f = 1 \rightarrow \log r_f = 0 \rightarrow \sigma(0) = 0.5$ (中性点)
- $r_f \rightarrow 0$ (sensor 不確か) $\rightarrow \log r_f \rightarrow -\infty \rightarrow K \rightarrow 0$
- $r_f \rightarrow \infty$ (sensor 高精度) $\rightarrow \log r_f \rightarrow +\infty \rightarrow K \rightarrow 1$

これで「信頼できる時ほど sensor を信じ、不確かな時ほど予測を信じる」という望ましい単調性が自動的に保証される。

要請 3: 2 自由度で柔軟性

$\log r_f$ をそのまま σ に入れる($\beta_0 = 0, \beta_1 = 1$)だけでも動くが、それは厳密に Bayesian variance-weighted blending と一致する固定形 になる(後述)。Project 1 は SPSA で task-specific な最適点を探すため、 β_0 (baseline)と β_1 (感応度)の 2 自由度を入れて Bayesian 解を一般化する。

4.3.2.7. β_0 と β_1 の各役割

$\beta_{0,f}$ は baseline gain:

$$K_f |_{r_f=1} = \sigma(\beta_{0,f})$$

$\beta_{0,f}$	$\sigma(\beta_{0,f}) = \text{baseline } K_f$	解釈
-3	0.05	sensor 強く不信(forward model 寄り)
-1.5	0.18	sensor 不信(C2 で qpos が学んだ値)
0	0.50	neutral(default)
+1.5	0.82	sensor 信頼
+3	0.95	sensor 強く信頼

$\beta_{1,f}$ は reliability への感応度:

$\beta_{1,f}$	動作
0	K は r_f に依存しない(常に $\sigma(\beta_0)$ で固定)
+0.5	緩やかな反応(default 設定)
+0.9	強い反応(C2 で qpos が学んだ値)
> 1	非常に強い反応(0/1 に張り付きやすい)
負	reliability 高 $\rightarrow K$ 低(逆方向)。生物学的に不自然なので SPSA は正に push する

具体例: $\beta_1 = 0.9$ で r_f が 10 倍になると $\log r_f$ は約 +2.3 増、 $\beta_1 \log r_f$ は +2.07 増 $\rightarrow \sigma$ の入力が大きく動き K が大きく動く。

4.3.2.8. 数値例: qpos field 軌道($\beta_0 = -1.5, \beta_1 = 0.9$ C2 学習値)

r_f	$v_f \approx \frac{1}{r_f}$	$\log r_f$	$\beta_0 + \beta_1 \log r_f$	$K_f = \sigma(\cdot)$
0.1	10.0	-2.30	-3.57	0.027
0.5	2.0	-0.69	-2.12	0.107
1.0	1.0	0.00	-1.50	0.182
2.0	0.5	+0.69	-0.88	0.293
10	0.1	+2.30	+0.57	0.639
100	0.01	+4.61	+2.65	0.934

innovation 分散が下がる(reliability 上がる)につれて K が滑らかに $0 \rightarrow 1$ へ。閾値ではなく soft switch になっている。

4.3.2.9. Bayesian inference との対応

2 つの Gaussian 情報源(精度 τ_{obs} と τ_{pred})を最適統合するときの観測重みは Bayesian 公式で:

$$w_{\text{obs}}^{\text{Bayes}} = \frac{\tau_{\text{obs}}}{\tau_{\text{obs}} + \tau_{\text{pred}}}$$

forward 予測の精度を baseline $\tau_{\text{pred}} = 1$ と置き、観測精度を $\tau_{\text{obs}} = r_f$ と読むと:

$$w_{\text{obs}}^{\text{Bayes}} = \frac{r_f}{r_f + 1} = \frac{1}{1 + \frac{1}{r_f}} = \sigma(\log r_f)$$

したがって $\beta_0 = 0, \beta_1 = 1$ のとき K_f は古典 Bayesian inverse-variance weighting と完全一致。Project 1 の logistic はこの Bayesian 解の一般化である。

自由度	Bayesian 解	Project 1 logistic 解
baseline	$\tau_{\text{pred}} = 1$ で固定	β_0 で自由 (SPSA で学習)
感応度	exactly 1	β_1 で自由

なぜ純粋 Bayesian にしないか:

1. forward model は perfect な Gaussian 観測者ではない (MyoSuite では bias / nonlinearity あり) $\rightarrow \tau_{\text{pred}} = 1$ に固定するのは強過ぎる前提。
2. v_f は真の variance ではなく EMA 推定の noisy proxy $\rightarrow$ 強くスケールしたくない場合がある ($\beta_1 < 1$)。
3. SPSA に task-specific な最適点 を探させたい $\rightarrow$ 2 自由度の方が表現力が高い。

4.3.2.10. 何が決まり、何が学習されるか

要素	固定 / 学習	備考
$\sigma(z)$ の関数形	固定	ロジスティック sigmoid を選択 (範囲 (0,1) + 滑らか + 微分可能)
log 変換	固定	reliability の指数スケールを線形化
$\beta_{0,f}$	SPSA で学習	baseline gain $\sigma(\beta_{0,f})$ の場所
$\beta_{1,f}$	SPSA で学習	reliability への感応度
$r_{f(t)}$	within-trial で動的更新	EMA から step ごとに変化
$K_{f(t)}$	毎 step 計算	r_f と β から決まる

したがって $K_{f(t)}$ は (i) 関数形と log 変換、(ii) SPSA が学習した 2 自由度、(iii) within-trial の reliability 動態、の 3 つの組合せで決まる。各 field f が独立に $\beta_{0,f}, \beta_{1,f}$ を持つので、qpos / qvel / act / tip_pos / reach_err それぞれが「自分の reliability の信じ方」を独自に学習できる。これが C2 で観察された field-wise specialization (qpos だけ β_0 が深く負、reach_err は $\beta_1 \approx 0$ で flat) を可能にする構造的選択である。

4.3.3. Across-trial layer: SPSA outcome adaptation

within-trial layer の挙動は $\beta = \{\beta_{0,f}, \beta_{1,f}\} \in \mathbb{R}^{10}$ に支配される。 β をどう決めるかは、reach 終了後の reaching outcome から学習する。

outcome は per-episode の minimum tip-to-target distance:

$$\text{minTip}(\beta) = \min_{t \in [0, T]} |p_{\text{tip}(t)} - p_{\text{tgt}}|$$

かみ砕いて言うと、1 回のリーチ動作の中で「指先がターゲットに一番近づいた瞬間の距離」を 1 つの数字にしたもの。たとえばコップに手を伸ばすとき、「いちばん近づいたとき何 cm 届かなかったか」がこの数字になる。

生体に置き換えると、リーチが終わった後に 目で見たり関節の感覚で「どのくらい届いたか」を感じ取った値に対応する。実際の脳がこの式の通り min を計算しているわけではないが、リーチ終了後に agent が自分で取得できる「結果」を 1 つの数字にまとめた 代表値 (operational proxy) として使う、というスタンス。

ポイント

1. minTip は agent 自身が 自分の感覚で取れる 値である (真の状態 x_t や oracle ラベルではない)。
2. 1 回のリーチに対して 1 つのスカラ が出る (複雑な軌道情報を圧縮)。
3. 「脳が literally このように計算している」とは主張しない。「リーチがどれくらいうまくいったか」の置き換えとして使うだけ。

4.3.3.1. なぜ最終位置でなく “最も近づいた距離” で評価するのか

ここで自然な疑問: 「リーチの結果」を測るなら episode 最終時刻の $|p_{\text{tip}(T)} - p_{\text{tgt}}|$ (final-tip) を使ってもよさそうなのに、なぜ全 step の最小 $\min_t |p_{\text{tip}(t)} - p_{\text{tgt}}|$ (min-tip) を選ぶのか? 理由は 4 つある。

1. リーチは「到達した時点で成功」が自然

人間がコップに手を伸ばすとき、目的は 指先がコップに届くこと。届いた瞬間に成功であり、その後手が少し戻ったり震えたりしても task の評価には関係ない。

- ・ final-tip: 一度びったり当てたあと手が震えて 2 cm 離れたら「2 cm の miss」と判定。一度も近づけなかった失敗試行と同じ扱いになり得る。
- ・ min-tip: 一度でも届けば「届いた」と評価される。届かなければ「届かなかった」。task 目的(=接触)と直接対応する。

2. MyoSuite の reach 設定には “止まる” 動作がない

シミュレーションは 12 秒(600 step)の固定長 episode。controller(joint-PD)は target qpos に向けて筋指令を出し続け、明示的な「停止」シグナルがない。

- ・ 早く target に到達した場合 → その後も 12 秒間筋活性を出し続ける → 筋粘性で微小振動
- ・ 最終 step ($t = T$)の位置は controller の収束 + 筋骨格振動で偶然決まる
- ・ 「リーチが成功したか」と「12 秒後にどこにいるか」は別の話

reach task では前者を測りたいので min-tip が適切。

3. Overshoot を二重カウントしない

筋骨格 controller では、reach 中盤で target を一度通り過ぎる(overshoot) ことがある。

時刻 t (s): 距離 (m)

0.0 : 0.40 (出発位置)

1.0 : 0.20

2.5 : 0.04 ← 最も近い瞬間 (min-tip)

3.0 : 0.06 (少し戻る、overshoot 後)

6.0 : 0.08 (落ち着く)

12.0 : 0.09 (final-tip)

- ・ min-tip = 0.04 → ピーク到達精度 を直接測れる
- ・ final-tip = 0.09 → 落ち着いた後の位置だが「controller が overshoot して戻ってきた」という余計な情報が混ざる

論文 C1-C4 は「reliability adaptation が 届く能力 に効くか」を問うので、controller 静定後のブレではなくピーク精度を評価したい。

4. paper では両方の metric を記録している

実際の評価では metrics.csv に複数の metric が同時に記録されている:

metric	意味
final_tip_error	最終 step($t = T$)の距離
min_tip_error	episode 中の最小距離(論文の headline metric)
max_tip_error	episode 中の最大距離(発散検出用)
overshoot	max - min(振動の振幅)
success_005	一度でも 5 cm 以内に入ったか(bool)
success_010	一度でも 10 cm 以内に入ったか(bool)
success_015	一度でも 15 cm 以内に入ったか(bool)

C1-C4 の headline は min-tip だが、final-tip も計算済みで Appendix で参照可能。reviewer に「final で評価したらどう違うか」と問われても回答できる構造になっている。

4.3.3.1.1. 生体研究との対応

人間 reach の運動神経科学でも、closest approach (CA) や hand velocity profile の peak / touch time といった min-tip 系の metric は伝統的に使われている。理由はここまでと同じ:

1. リーチの目的は ターゲットに到達すること(touch / grasp の前段)
2. 一度到達した後の手の動きは別問題(hold / grasp / reset 等)
3. 視覚 + proprioception で「届いた感覚」を得るのは closest approach の瞬間

つまり min-tip は生体研究との対応も悪くない。

4.3.3.1.2. final-tip を使うべき他の task(反例)

逆に、final-tip が望ましい場合もある:

task	評価したい性質	適切な metric
holding	target 位置で静止して保持	final-tip + variance
tracking	連続的に target を追跡	time-averaged tracking error
end-state matters	終了時点が評価される	final-tip(or final-state)

これらは「一度通れば OK」ではなく「最後にどこにいるか」が直接 task definition の一部。reach task はこのカテゴリに該当しないので min-tip が適切。

まとめ

$\min_t |p_{\text{tip}(t)} - p_{\text{tgt}}|$ を選ぶ理由:

1. task 目的(届くこと)と直接対応
2. 12 秒固定 episode で controller が停止しない事情の回避
3. overshoot をピーク精度と分離して測れる
4. 生体 reach 研究の古典的 metric と整合

4.3.3.2. どうやって β を更新するか — 勾配が取れない問題

理想的には、 β を minTip が小さくなる方向に少しずつ動かしたい。これは機械学習で言う 勾配降下法 で、入力 (β) を 1 単位動かしたら出力 (minTip) がどれだけ変わるかという 勾配 を計算して、その逆方向に β を更新する手法。

ところが Project 1 の設定では、この勾配を 直接計算する道がない。理由を順に並べると次のようになる。

なぜ計算できないか	中身
① 計算の流れが長い	β から minTip までの道は、(a) 観測 → (b) EMA で reliability 計算 → (c) sigmoid で K に変換 → (d) joint-PD で筋指令を出す → (e) MuJoCo で 600 step 物理シミュレーション → (f) 軌道から最小距離を取る、という長い手順。途中で勾配を 1 つでも切ると全体の勾配が取れない。
② シミュレータが “ブラックボックス”	MuJoCo は内部で剛体力学・接触・筋粘性を解く物理計算。これ自体は微分可能なように作られていないので、 β がほんの少し変わったときに 600 step 後の指先位置がどう変わるかを数式で書き下せない。
③ “最小を取る” が滑らかでない	$\min_t \cdot $ という操作は数学的に微分が連続的に定義できない。たとえば「2 回目の山と 3 回目の山のどちらが小さいか」が β のわずかな変化で入れ替わる瞬間、勾配が跳ねる。
④ 結果が確率的に揺れる	target index や noise の seed が違うと同じ β でも minTip の値が違う。仮に勾配が定義できて、毎回ノイズで上下する。

要するに、 β から minTip までは長い計算経路 + 微分できない最小化 + 確率的揺らぎ が重なっていて、教科書的な勾配降下は使えない。

そこで Project 1 は SPSA(Simultaneous Perturbation Stochastic Approximation, Spall 1992) を使う。SPSA は勾配を数式から求めるのではなく、 β を少し + 方向と - 方向に揺らして、結果がどちらでより小さくなるかだけを見て、 β を更新する 方式。

直感

β をパラメータ “つまみ” と思って、少しひねって試して、結果が良くなる方にひねる を繰り返すイメージ。中身を理解する必要がなく、結果だけ見れば動く。

SPSA はその “少しひねる” を 10 個のつまみ全部同時に行う巧妙な方式で、1 回の試行で全 10 個のつまみの調整方向を一度に推定できる(後述)。

SPSA 1 iteration:

```
Delta_n ~ uniform({-1, +1})^10 // Rademacher perturbation
o_plus = minTip(beta_n + c_n * Delta_n) // S 個 episode 平均
o_minus = minTip(beta_n - c_n * Delta_n) // S 個 episode 平均
g_hat = (o_plus - o_minus) / (2 c_n) * Delta_n^{-1}
beta_{n+1} = beta_n - a_n * g_hat
```

Spall schedule:

```
a_n = a / (n + 1 + A)^alpha_s
c_n = c / (n + 1)^gamma_s
```

defaults: a=2.0, c=0.3, alpha_s=0.602, gamma_s=0.101, A=5, S=10 or 12。

Black-box stochastic approximation

最適化対象の中身を見ずに、入力を摂動して出力(scalar evaluation)を観測するだけで勾配推定する手法群。SPSA はそのうち最も次元スケーラブルなアルゴリズム。policy search / Evolution Strategies と同じカテゴリ。

Project 1 では SPSA を 2 つの設定で走らせた:

variant	training cells	使い道
single-cell	($\sigma = \text{none}, d = 18$) の 1 cell のみ	agent-available adaptation が closed-loop oracle に追いつけるかの存在証明 (C2)
full-grid	3 noise $\times$ 2 delay = 6 cell から uniform random sampling	single global β で grid を覆えるかの structural test (C3)

4.3.3.3. SPSA の詳細解説

ここでは Project 1 で多用されている SPSA (Simultaneous Perturbation Stochastic Approximation) の原理を、stochastic approximation の系譜から SPSA 独自の摂動戦略、収束理論、実装上の注意点まで詳しく説明する。

4.3.3.3.1. 背景: なぜ勾配降下を直接使えないか

目的関数 $J(\beta) = \mathbb{E}[\text{minTip}(\beta)]$ を最小化したい。標準的な勾配降下は

$$\beta_{n+1} = \beta_n - a_n \nabla_{\beta} J(\beta_n)$$

を回す。しかし Project 1 の設定では $\nabla_{\beta} J$ を解析的に取れない:

1. outcome $\text{minTip}(\beta)$ は env step $\rightarrow$ EMA $\rightarrow$ sigmoid $\rightarrow$ joint-PD control $\rightarrow$ 非可微の $\min_t |\cdot|$ を経由する長い計算グラフを通る。
2. env が物理シミュレーション(MuJoCo)で、解析的微分が事実上不可能。
3. target は episode ごとに変わり、 J は確率的に noisy(評価のたびに値が違う)。

つまり目的関数を black box として扱う必要がある。

4.3.3.3.2. Stochastic approximation の系譜

stochastic approximation は Robbins–Monro (1951) に始まる古典的最適化手法の族で、noisy な関数評価から最適点を逐次的に追っていく 枠組み。代表的アルゴリズム:

手法	何を観測	勾配推定の方法
Robbins-Monro (1951)	scalar $J(\beta) + \text{noise}$	理論的枠組み(具体的アルゴリズムは指定なし)
Kiefer-Wolfowitz (1952)	$J(\beta + h e_i), J(\beta - h e_i)$ for each i	有限差分: 1 次元あたり 2 評価、 p 次元なら $2p$ 評価/iter

SPSA (Spall 1992)	$J(\beta + c\Delta), J(\beta - c\Delta)$ where $\Delta \in \{\pm 1\}^p$	全次元を 1 つの Rademacher 摂動で同時推定: 常に 2 評価/iter
Evolution Strategies	$J(\beta + \sigma Z_k)$ for $k = 1..K$	複数 Gaussian 摂動の reward-weighted 平均

SPSA は Kiefer-Wolfowitz の有限差分と Evolution Strategies の中間。“差分” は使うが次元数に依存せず常に 2 評価で済む。

4.3.3.3.3. Rademacher 摂動とは

SPSA の中心道具である Rademacher 摂動 を先に説明しておく。

定義: Rademacher distribution

スカラー確率変数 X について、 $X = +1$ と $X = -1$ をそれぞれ確率 $\frac{1}{2}$ で取るとき、 X は Rademacher 分布 に従うという(ドイツの数学者 Hans Rademacher にちなむ)。

言い換えると: フェアなコイン投げで「表 = +1、裏 = -1」とした値そのもの。

統計的性質:

量	値
期待値 $\mathbb{E}[X]$	0 (対称分布)
分散 $\text{Var}(X)$	1
絶対値 $ X $	常に 1 (値域 $\{-1, +1\}$)
逆数 $\frac{1}{X}$	常に ± 1 で有限 ($X = 0$ にならない)

特に重要なのは 逆数が有限であること。後述の SPSA 勾配推定で $\frac{1}{\Delta_i}$ を使うので、 Δ_i が 0 になり得る分布 (Gaussian など)は使えない。

Rademacher 摂動ベクトル は、これを p 次元に拡張したもの:

$$\Delta = (\Delta_1, \Delta_2, \dots, \Delta_p) \in \{-1, +1\}^p$$

ただし 各成分 Δ_i は独立 に ± 1 を等確率で取る。Project 1 では $p = 10$ (β の次元)。

具体例 ($p = 10$):

Delta example 1: (+1, -1, +1, +1, -1, -1, +1, -1, +1, +1)

Delta example 2: (-1, -1, +1, -1, +1, +1, -1, +1, -1, -1)

Delta example 3: (+1, +1, -1, +1, -1, +1, +1, -1, -1, +1)

このベクトルを使って β を「全成分同時に $\pm c$ で揺らす」のが SPSA の核心。

直交性 (orthogonality) という重要な性質を満たす。

(i) 各成分の平均は 0:

$$\mathbb{E}[\Delta_i] = 0$$

(ii) 独立性 と $\Delta_i^2 = 1$ より:

$$\mathbb{E}[\Delta_i \Delta_j] = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

(iii) 異なる成分の比の期待値が 0 (SPSA 不偏性の核):

$$\mathbb{E}[\Delta_j / \Delta_i] = \mathbb{E}[\Delta_j] \cdot \mathbb{E}[1 / \Delta_i] = 0 \quad \text{for } i \neq j$$

4.3.3.3.4. この (iii) がなぜ SPSA を成立させるのか

ここがいちばん分かりにくいので、ゆっくり読み解く。

そもそも何が嬉しいのか?

SPSA は 10 個のパラメータ $\beta_1, \dots, \beta_{10}$ をいっぺんに揺らして、たった 2 回の試行で全 10 個の「動かす方向」を推定する。常識的には「同時に揺らしたら、どの効果がどのパラメータのせいかわからなくなるはず」。(iii) はその直感に反して混ざってしまった効果が長期的には自動で分離されることを保証する。

アナロジー: 10 人で同時に綱を引く

10 本の綱が机の上であって、それぞれ少しずつ違う方向に何かを動かす。あなたは 10 本の綱を 1 本ずつ引いて効果を測るのが理想だが、時間がないので全 10 本を同時にランダムに引いたり緩めたり(各々 +1 か -1)して、机に伝わった全体の動き 1 つだけを観測する。

驚くべきことに:

- ・ i 番目の綱の効果を推定したいとき、「観測した全体の動き $\times i$ 番目で何方向に引いたか」を計算する
- ・ この量は「 i 番目の効果」+「他 9 本の効果がたまたま混ざったもの」になる
- ・ 後者(混ざった分)は試行を繰り返すと平均でちょうどゼロになる

これが Rademacher の直交性 (iii) の意味。

なぜ平均でゼロになるか

10 番目の綱の効果が i 番目の推定にどれだけ混ざるかは、 Δ_j/Δ_i という比に比例する。これは独立な ± 1 の比なので、毎回 +1 か -1 を等確率で取る:

trial 1: $\Delta_j = +1, \Delta_i = +1 \rightarrow$ 比 = +1

trial 2: $\Delta_j = -1, \Delta_i = +1 \rightarrow$ 比 = -1

trial 3: $\Delta_j = +1, \Delta_i = -1 \rightarrow$ 比 = -1

trial 4: $\Delta_j = -1, \Delta_i = -1 \rightarrow$ 比 = +1

...

長期的にはこの +1 と -1 が等しく現れるので、平均 = 0。これが (iii) の式が言っている内容。

推定式に当てはめると

SPSA の 1 回の勾配推定 $\hat{g}_i$ は実は次のような構造をしている:

$$\hat{g}_i = \underbrace{\frac{\partial J}{\partial \beta_i}}_{\text{本当に欲しい成分}} + \underbrace{\sum_{j \neq i} \frac{\partial J}{\partial \beta_j} \frac{\Delta_j}{\Delta_i}}_{\text{他成分から混ざったノイズ}}$$

直交性 (iii) より、ノイズ部分の期待値が 0 なので:

$$\mathbb{E}[\hat{g}_i] = \frac{\partial J}{\partial \beta_i}$$

つまり長期的に平均すれば、SPSA は正しい勾配を返す(統計学で言う「不偏推定量」になる)。

覚え方

1 回の SPSA 推定は noisy だが、向きは正しい方向に偏っている。

反復することでノイズが打ち消し合い、少しずつ正しい勾配方向に進む。

これは「揺さぶってみて結果が良くなった方に進む」という直感的な探索の、数学的に正当化された実装になっている。

4.3.3.3.5. SPSA の中心アイデア: simultaneous perturbation

10 次元の $\beta = (\beta_1, \beta_2, \dots, \beta_{10})$ の勾配を有限差分で取るなら、各成分について

$$\frac{\partial J}{\partial \beta_i} \approx \frac{J(\beta + h e_i) - J(\beta - h e_i)}{2h}$$

を計算するので 20 評価必要(成分ごとに $\pm$ 方向で 2 評価)。これを全 10 成分同時に摂動して 2 評価で全部やる、というのが SPSA。

具体的に、Rademacher 摂動ベクトル $\Delta \in \{-1, +1\}^{10}$ を 1 つサンプルして

$$J(\beta + c\Delta), \quad J(\beta - c\Delta)$$

を評価する(c は摂動幅、Spall schedule で減衰する小さな正数)。そして i 番目の勾配推定 を次式で計算する:

$$\hat{g}_i = \frac{J(\beta + c\Delta) - J(\beta - c\Delta)}{2c\Delta_i}$$

要素的に書くと:

$$\hat{g} = \left(\frac{J(\beta + c\Delta) - J(\beta - c\Delta)}{2c} \right) \cdot \begin{pmatrix} 1/\Delta_1 \\ 1/\Delta_2 \\ \vdots \\ 1/\Delta_{10} \end{pmatrix}$$

つまり 1 つのスカラー差分 $\frac{J^+ - J^-}{2c}$ を、摂動方向の逆数ベクトル $\Delta^{-1} = \left(\frac{1}{\Delta_1}, \dots, \frac{1}{\Delta_{10}} \right)$ で broadcast している。Rademacher なので $\frac{1}{\Delta_i} = \Delta_i$ 、実装上は要素積になる。

この推定の見た目

$\hat{g}_i$ には i 番目以外の成分の勾配寄与も混ざっている(下記の展開で出る)。

にもかかわらず期待値を取ると正しい勾配になる、というのが Spall (1992) の発見。Rademacher の直交性 $\mathbb{E}\left[\frac{\Delta_j}{\Delta_i}\right] = 0 \ (j \neq i)$ によりクロス項が期待値で消える。

4.3.3.3.6. なぜこの推定が動くのか

J を Taylor 展開:

$$J(\beta \pm c\Delta) = J(\beta) \pm c \sum_i \frac{\partial J}{\partial \beta_i} \Delta_i + \left(\frac{c^2}{2} \right) \sum_{i,j} \frac{\partial^2 J}{\partial \beta_i \partial \beta_j} \Delta_i \Delta_j \pm O(c^3)$$

差分を取ると一次項だけ残る:

$$J(\beta + c\Delta) - J(\beta - c\Delta) = 2c \sum_i \frac{\partial J}{\partial \beta_i} \Delta_i + O(c^3)$$

これを $2c\Delta_i$ で割って i 成分目の推定とする:

$$\hat{g}_i = \sum_j \frac{\partial J}{\partial \beta_j} \frac{\Delta_j}{\Delta_i} + O(c^2) = \frac{\partial J}{\partial \beta_i} + \sum_{j \neq i} \frac{\partial J}{\partial \beta_j} \frac{\Delta_j}{\Delta_i} + O(c^2)$$

第 2 項(クロス項)が重要で、Rademacher の性質より:

$$\mathbb{E}\left[\frac{\Delta_j}{\Delta_i}\right] = \mathbb{E}[\Delta_j] \cdot \mathbb{E}\left[\frac{1}{\Delta_i}\right] = 0 \quad (j \neq i)$$

つまり 他の成分の勾配寄与は期待値 0 になるので、

$$\mathbb{E}[\hat{g}_i] = \frac{\partial J}{\partial \beta_i} + O(c^2)$$

SPSA の核心

Rademacher 摂動 $\Delta \in \{-1, +1\}^p$ は直交性 $\mathbb{E}\left[\frac{\Delta_j}{\Delta_i}\right] = 0 \ (j \neq i)$ を持つ。

これにより、1 つの摂動方向 Δ で全 p 成分の勾配を同時に 不偏推定 できる。

4.3.3.3.7. なぜ Rademacher で Gaussian ではないか

$\Delta_i \sim N(0, 1)$ (Gaussian) でも展開はできる。が、SPSA は Rademacher を選ぶ:

	Rademacher ± 1	Gaussian $N(0, 1)$
$\frac{1}{\Delta_i}$ の分散	常に 1(有限)	∞ (0 近傍で発散)
推定の variance	bounded	unbounded $\rightarrow$ 不安定

外れ値のリスク	なし(全要素 $ \Delta_i = 1$)	大きな摂動で実装が壊れる可能性
計算量	$\Delta_i^{-1} = \Delta_i$ で済む	割り算が必要

Spall (1992) は Rademacher を含む bounded inverse moment を持つ分布を理論的に正当化した。実用上はほぼ常に Rademacher。

4.3.3.3.8. Spall schedule の意味

学習率 a_n と摂動幅 c_n は iteration n について減衰する:

$$a_n = \frac{a}{(n+1+A)^{\alpha_s}} \quad c_n = \frac{c}{(n+1)^{\gamma_s}}$$

parameter	default	役割
a	2.0	初期学習率の規模
c	0.3	摂動幅の規模
A	5	初期 iteration の “stability buffer” (初期の大きな step を抑える)
α_s	0.602	学習率減衰指数 ($a_n \rightarrow 0$ as $n \rightarrow \infty$)
γ_s	0.101	摂動減衰指数 ($c_n \rightarrow 0$ as $n \rightarrow \infty$)

Spall (1992) の収束定理が要求する条件は:

$$\sum_n a_n = \infty, \quad \sum_n \frac{a_n^2}{c_n^2} < \infty, \quad \sum_n a_n c_n^2 < \infty$$

$\alpha_s = 0.602, \gamma_s = 0.101$ という値はこれらの条件をギリギリ満たしつつ実用的に速い収束を与える (Spall 推奨)。

数値例(初期と後期):

n	a_n	c_n	挙動
0	0.69	0.30	大きな step、広い摂動で global 探索
10	0.40	0.24	調整段階
50	0.20	0.18	収束途上
99	0.13	0.16	小さな step、狭い摂動で局所微調整

4.3.3.3.9. Paired perturbation と分散低減

Project 1 では samples_per_side $S \in \{10, 12\}$ episode の平均を取る:

$$o_n^+ = \frac{1}{S} \sum_{s=1}^S \text{minTip}(\beta_n + c_n \Delta_n, \text{seed}_{n,s,+})$$

$$o_n^- = \frac{1}{S} \sum_{s=1}^S \text{minTip}(\beta_n - c_n \Delta_n, \text{seed}_{n,s,-})$$

ここで paired というのは、 s ごとに同じ target index と同じ env seed を使って $\beta + c\Delta$ と $\beta - c\Delta$ の両方を評価することを意味する(common random numbers)。これで $o_n^+ - o_n^-$ の分散が大きく減る:

$$\text{Var}(o^+ - o^-) = \text{Var}(o^+) + \text{Var}(o^-) - 2 \text{Cov}(o^+, o^-)$$

paired にすれば o^+ と o^- は同じ noise pattern を共有するので $\text{Cov}(o^+, o^-) > 0$ となり、差の分散が減る。Project 1 では target index と initial seed を + と - で一致させて paired evaluation を実現している。

4.3.3.3.10. Project 1 での 1 iteration コスト

1 SPSA iteration あたりの env episode 数:

$$2 \cdot S = 2 \cdot 10 = 20 \text{ episode} \quad (\text{single-cell}, S = 10)$$

$$2 \cdot S = 2 \cdot 12 = 24 \text{ episode} \quad (\text{full-grid}, S = 12)$$

100 iteration で:

variant	iter 数	episode 数	walltime (CPU, M1 Pro)
single-cell ($S = 10$)	100	2000	~ 30 分
full-grid ($S = 12$)	100	2400	~ 2 時間(6 cell uniform)

有限差分なら同等精度に $10 \text{ 次元} \times 2 \cdot S = 200$ 評価/iter で 1 桁多くかかる。SPSA はこの 1 桁分のコスト圧縮で次元数によらず常に 2 評価/iter を達成している。

4.3.3.3.11. 収束の直感

1 iteration あたりの勾配推定はかなり noisy(他成分のクロス項が単独 sample では消えない)。が、SPSA は:

1. 各 iteration で 向きは正しい方向に確率的に動く (不偏性)
2. Spall schedule で 学習率を段階的に絞る
3. 学習率の二乗和は有限 なので、長期的には収束する

結果として 100 iteration 程度で 10 次元の β が安定する。Project 1 の C2(F_spsa_single)で観察される下降軌道は:

- ・ 初期 ($n < 20$): 大きな揺れ、向きを探索
- ・ 中期 ($20 \leq n \leq 60$): outcome が下がる方向に bias がついて改善
- ・ 後期 ($n > 60$): 微調整、ほぼ収束

4.3.3.3.12. SPSA の限界と Project 1 での意味

SPSA は次元スケールブルだが万能ではない:

限界	Project 1 での影響
局所最小	$J(\beta)$ が非凸なら局所最小に捕まる可能性。Project 1 では β 空間が比較的なめらかなので問題化していないが、context-conditioned β network (Project 1.5) に拡張すると深刻化する可能性
hyperparameter 選択	a, c, A の選択は task 依存。Project 1 は Spall 推奨値を採用しているが、収束速度は task ごとに改善余地
評価コスト	1 iteration = $2S$ episodes は env が安価なら問題ないが、real robot では実用的でない場合がある
低次元向き	$p \geq 100$ 次元では SPSA より policy gradient / ES の方が分散が低い場合がある。Project 1 は $p = 10$ なので SPSA が最適

Project 1 が SPSA を選んだ理由

1. 10 次元 $\rightarrow$ SPSA の dimension scalability の恩恵が最大
2. outcome が non-differentiable(min over t) $\rightarrow$ gradient 計算不可
3. 評価は env simulation で十分高速 $\rightarrow$ 1 iter ~ 20 episode 許容
4. Spall schedule の確率収束保証が信頼できる

これらが揃っているため、Project 1 では SPSA が agent-available outcome-driven adaptation の自然な実装となる。

4.3.4. oracle-supervised predictor: upper bound diagnostic (Appendix C 教材)

提案手法と対比される旧 main contribution(現 Appendix C)は、condition feature (σ, d, c) から scalar K を出す MLP を、per-condition の K -sweep oracle ラベルで supervised に学習する。

$g_phi : (one_hot(c), \sigma, d/d_max) \rightarrow K \text{ in } [0,1]$

oracle ラベルは 2 種類:

oracle	定義	最適化するもの
K^*_{ol}	$\argmin_K E[\hat{x}_t(K) - x_t ^2]$	open-loop state estimation error
K^*_{cl}	$\argmin_K E[\min Tip(K)]$	closed-loop reaching error

Appendix C 教材としての位置

oracle-supervised predictor は agent-available ではない: per-condition の K-sweep を回す段階で生体には不可能な data 収集を要求する。提案手法と直接比較するための non-agent-available upper bound として Appendix C に残してある。

重要な観察: closed-loop oracle K_{cl}^* と open-loop oracle K_{ol}^* は乖離することがあり、特に multi-step supervision された forward model 下で顕著(closed-loop で $K = 0$ が optimal なのに、open-loop oracle では $K = 1$ が optimal)。これは learned correction gain を評価する際に 何を教師 oracle としたかを明示すべきだという、Appendix C の中心的指摘である。

5. 実験 pipeline

5.1. Repository の構造

src/myoarm_fse/ Python package
scripts/ CLI scripts
configs/ YAML configs
runs/ local outputs large artifacts are not versioned
tests/ unit and smoke tests
docs/ plans, primers, this textbook
figures/ paper figures and CSV summaries
paper/ TNNLS manuscript

環境構築とテストは uv を使う。

```
uv sync
uv run pytest
uv run pytest -m myosuite
```

落とし穴

Python を使う場合は python 直実行ではなく uv run python ... を使う。依存関係と仮想環境を固定するためである。

5.2. 最小再現手順

研究を最初から再現する場合の概念的な手順は以下である。実際のファイル名は configs/ と scripts/ を確認する。

Step	作業	確認する出力
1	target set 生成	runs/targets/*.npz
2	episode collection	runs/episodes/<timestamp>/
3	TransitionDataset 作成 / concat	runs/datasets/*.npz
4	forward model training	runs/models/<timestamp>/
5	open-loop estimator sweep	runs/estimators/<timestamp>/metrics.csv
6	learned gain training	runs/learned_gain_models/<timestamp>/
7	closed-loop evaluation	runs/closed_loop/<timestamp>/metrics.csv
8	paper figures	figures/F*.png, figures/data/*.csv

5.3. 実験 command の読み方

代表的な command は次の形になる。

```
uv run python scripts/train_forward_model.py \
--config configs/models/mlp_expanded.yaml

uv run python scripts/evaluate_estimator.py \
--config configs/estimators/fixed_kalman_robustness.yaml \
```



```
--forward-model runs/models/<model-id>
```

```
uv run python scripts/evaluate_closed_loop.py \
--config configs/closed_loop/<config>.yaml
```

設定 YAML を変えるときは、実験条件を run directory に保存し、後で paper figure を再生成できるようにする。

6. 主要結果の読み方

ここまでで道具(forward model、 predictive state observer、 within-trial / across-trial の二層適応、 Appendix C の oracle-supervised 教材)が揃った。ここからはそれらを使って論文の主張 C1～C4 を読む。

6.1. 4 つの主張をひとことで

Claim	何が分かったか	一言要約
C1	初期設定の reliability ルール($\beta_0 = 0, \beta_1 = 0.5$)では、6 つの cell すべてで 目標 $K = 0$ から 23～29 cm 離れた min-tip が出る。	innovation を眺めるだけでは「センサーを信じるか予測を信じるか」の正しい比率にたどり着けない。
C2	1 つの cell に絞って SPSA を 100 回まわすと、その cell で 23 cm の差のうち約 17 cm を埋める($0.77 \text{ m} \rightarrow 0.56 \text{ m}$, $K = 0$ は 0.50 m)。	リーチの結果(minTip)1 つを見るだけで、agent は自分で gain ルールを調整できる。
C3	6 cell から毎回ランダムに 1 つ選んで SPSA を回しても、delay 18 の 3 cell でわずか 3～5 cm の改善、delay 0 の 3 cell では改善ゼロ。	SPSA の問題ではなく、 β を 10 個の数だけで表現する設計の限界。cell ごとに違う β を出す仕組みが必要。
C4	ReachFixed で学んだ β を ReachRandom にそのまま持ってくると悪化する($K = 0$ を 4～5 cm 下回る)。一方、初期設定 reliability の方は意外にも拮抗する。	正しい gain ルールはタスク次第。学習結果は別のタスクには移植できない場合がある。

6.2. C1 — 初期設定の reliability ルールはタスクに合わない

6.2.1. どんな実験か

二層 adaptation の 外側のループ(SPSA)を一切回さず、 within-trial ルールだけを default 設定($\beta_{0,f} = 0$, $\beta_{1,f} = 0.5$)で動かし、結果の min-tip を 6 cell すべてで測った。

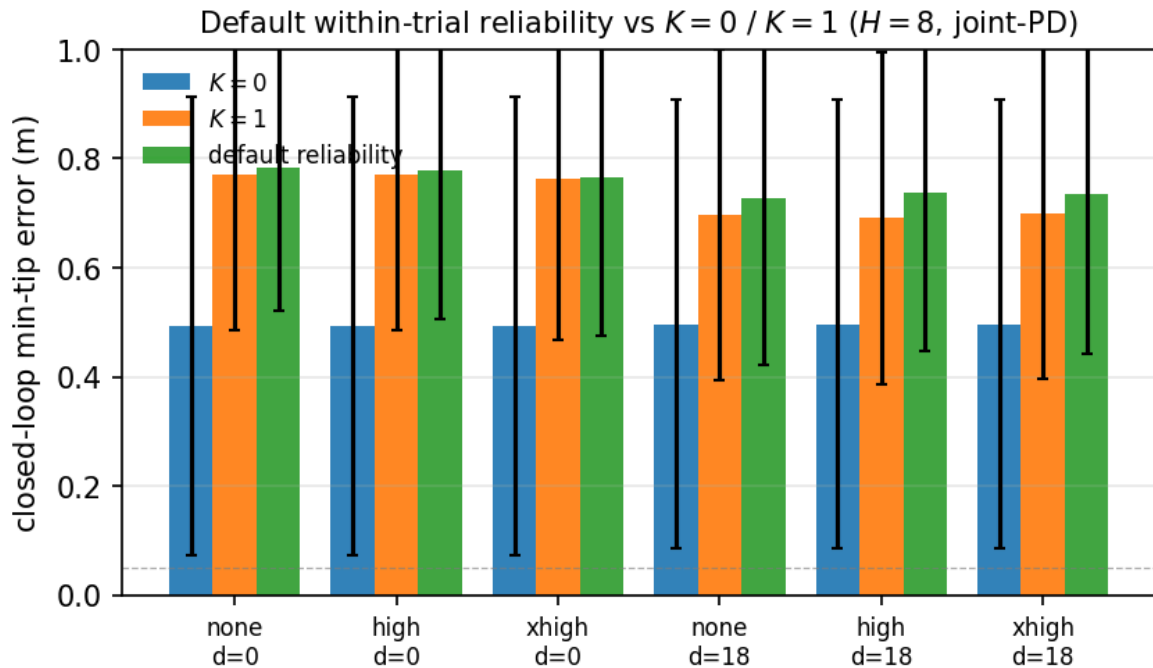


図 4: 青 = $K = 0$ (forward model だけ信じる)、橙 = $K = 1$ (センサーだけ信じる)、緑 = default reliability。default は $K = 0$ を 23～29 cm 下回る。

6.2.2. 何が起きているか — 実測の per-field K_f

default ルールが各 cell で出す per-field の correction gain K_f を、1 episode 走らせて取得した結果:

cell	qpos	qvel	act	tip_pos	reach_err
none, d=0	0.96	0.49	0.91	0.97	0.98
none, d=18	0.70	0.31	0.69	0.59	0.68
high, d=0	0.96	0.49	0.92	0.96	0.97
high, d=18	0.69	0.32	0.71	0.68	0.69
xhigh, d=0	0.90	0.45	0.91	0.94	0.94
xhigh, d=18	0.67	0.31	0.69	0.64	0.68

読み取れる事実:

1. qvel(関節速度)だけが一貫して低い gain(0.31~0.49)。
2. それ以外の 4 field は中~高めの gain(0.59~0.98)に張り付く。
3. どの cell でも $K_{cl}^* = 0$ (センサーを 100% 無視するのが正解)なのに、default ルールはそこから遠く離れている。

6.2.3. なぜ届かないか

ルールの形が原因。 $K_f = \sigma(\beta_{0,f} + \beta_{1,f} \log r_f)$ で、 $\beta_{0,f} = 0$ のとき baseline は $\sigma(0) = 0.5$ 。ここから K_f をほぼ 0 まで下げるには、 $\log r_f$ が大きな負の値(=信頼度が極端に低い、つまり innovation が極端に大きい)になる必要がある。 H=8 forward model は十分正確なので innovation はそこまで大きくならず、 K_f は中~高に張り付いたまま。

C1 の核心

innovation を見るだけでは、 forward model が「タスクの目的に対して」どれくらい役立つかは分からない。

- ・ innovation は「センサーとの予測ズレ」を教えてくれる。
- ・ でも「予測でリーチが届くかどうか」は、リーチが終わって 結果(minTip)を見るまで 分からない。

つまり within-trial layer 単独では、タスクが要求する gain 水準(この場合 $K = 0$)に到達する手段がない。

6.3. C2 — 1 つの cell で SPSA を回すと、差はほぼ埋まる

6.3.1. どんな実験か

最も難しい 1 cell($\sigma = \text{none}$, $d = 18$ ステップ = 0.36 秒の遅延)に 固定して、SPSA を 100 iteration 走らせて β を学習させた。 各 iteration は 10 episode $\times$ ± 2 セット = 20 episode。

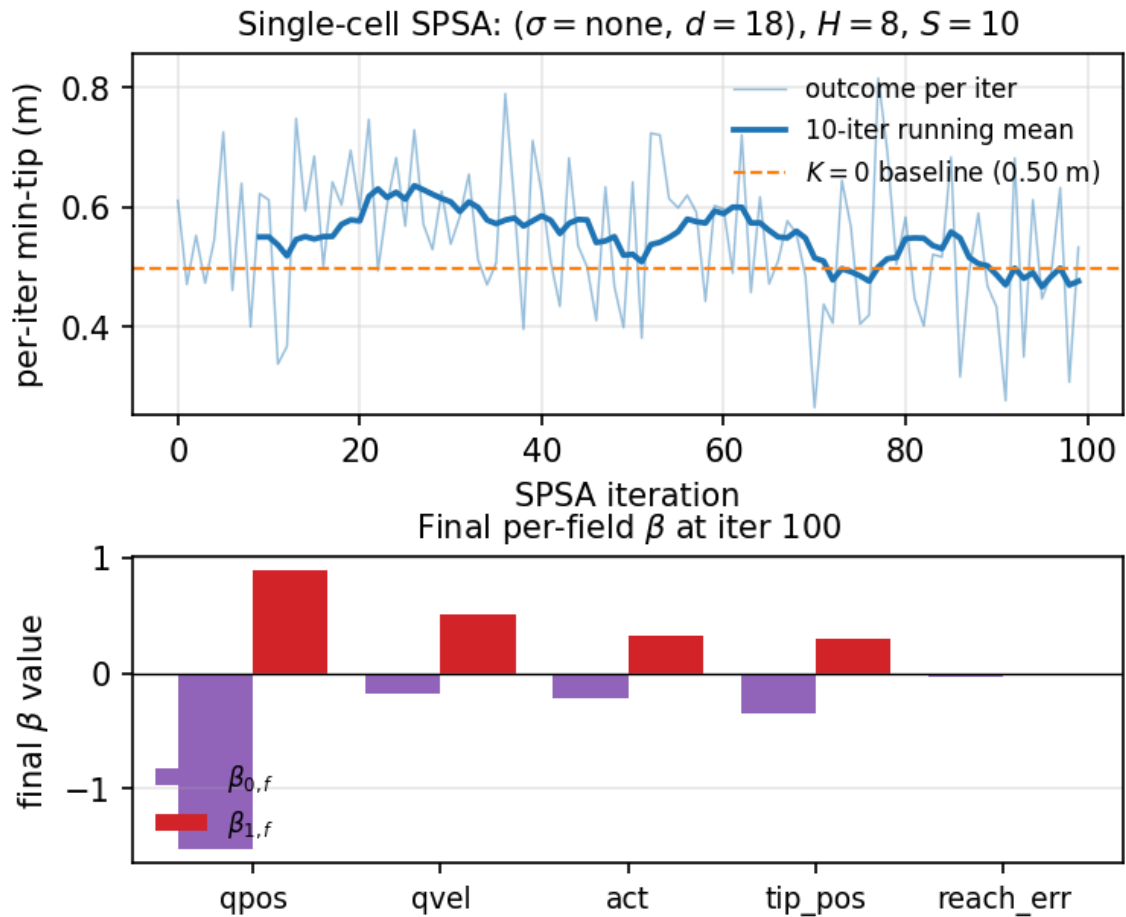


図 5: 上: SPSA を回すごとに minTip(=リーチ結果)がどう変わったか。細い青線が 1 iter ごとの値、太い濃青が 10 iter の移動平均、橙破線が $K = 0$ baseline(0.50 m)。下: 100 iter 後の per-field β 。qpos が intercept(下方修正)も slope(感度)も先頭。

6.3.2. 数字の読み方

- ・ 1 iter ごとの outcome: 初期 ≈ 0.61 m $\rightarrow$ 最終 iter ≈ 0.53 m
- ・ 10-iter 移動平均(訓練ノイズを均した値): 一時的に $K = 0$ baseline の 0.50 m に到達
- ・ 学習済み β を別の 10 episode で改めて評価(deployed eval): 0.56 m
- ・ 元の default ルール: 0.77 m
- ・ 差し引き: $0.77 \rightarrow 0.56$ で 21 cm 改善。 $K = 0$ までの残差 6 cm。

6.3.3. どこに「学習」が現れたか

final_beta.json を見ると、5 field の β がそれぞれ異なる方向に動いている。特に qpos は intercept $\beta_{0,\text{qpos}} \approx -1.5$ 、slope $\beta_{1,\text{qpos}} \approx +0.9$ と他 field より際立つ。これは「関節位置センサーの信頼度は通常時は強く抑え、innovation が増えたら敏感に反応せよ」という field-wise specialisation が outcome 駆動で自動的に出現したことを意味する。

C2 の核心

リーチの結果(minTip)を見るだけで、agent は自分の gain ルールを正しく調整できる。

- ・ 与えられる情報: per-episode で 1 個のスカラー(届いた距離の最小値)。
- ・ それだけから 10 次元の β 全部を SPSA で更新する。
- ・ 結果: default では 0.77 m だったリーチが 0.56 m に。 $K = 0$ という「センサーを使うな」という極端な正解とほぼ同等。

教師信号(=「最適 K はこれ」というラベル)を外から与えられない設定で、agent が自力で改善できることが分かった。

6.4. C3 — 6 cell 全部を相手にすると、改善は頭打ちになる

6.4.1. どんな実験か

C2 では 1 cell に固定したが、6 cell から毎 iteration でランダムに 1 つを選んで SPSA を回した。1 つの β が 6 cell すべてに対応できるかをテスト。

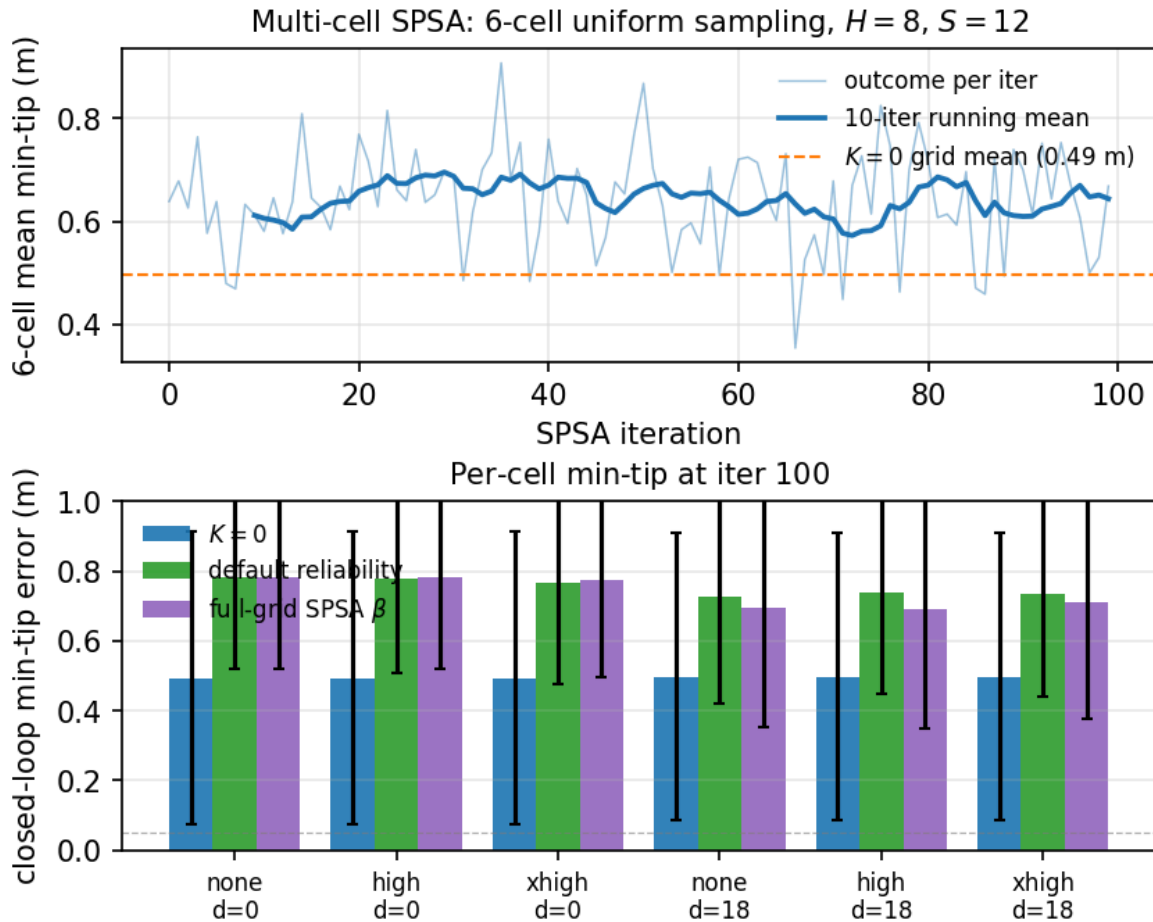


図 6: 上: 6 cell の平均 minTip の SPSA 軌跡。 $K = 0$ の 6 cell 平均(0.49 m)に届かない。 下: 100 iter 後の各 cell の minTip を、 $K = 0$ (青)・default reliability(緑)・SPSA 学習後 β (紫)で比較。

6.4.2. 数字の読み方

- ・ delay 18 の 3 cell: default に対し 3~5 cm の改善。ただし $K = 0$ までは依然 19~22 cm 離れている。
- ・ delay 0 の 3 cell: SPSA の改善はほぼゼロ。 $K = 0$ までは ~28 cm 離れている。
- ・ どの cell も $\mathbf{K}_{cl}^* = \mathbf{0}$ が正解なのに、1 つの β ではそこまでたどり着けない。

6.4.3. なぜ頭打ちになるか — parameterisation の限界

β は実数 10 個分($\beta_{0,f}$ 5 個 + $\beta_{1,f}$ 5 個)。これだけで 6 つの環境すべての最適点を表現することはできない。

具体例:

- ・ 高ノイズ cell では innovation が大きいので K_f が小さくなりやすい → そのまま $K_f \approx 0$ に近づける $\beta_{0,f}$ を選びたい
- ・ 低ノイズ cell では innovation が小さいので K_f が大きくなりやすい → 上と同じ $\beta_{0,f}$ では over-suppress(下げ過ぎ)になる

1 つの β で両立しないので、SPSA は妥協点に収束する。これは SPSA という最適化手法の失敗ではない (C2 で 1 cell なら届く)。「innovation の情報だけ受けて β を出す」という設計の限界。

C3 の核心 — SPSA が悪いのではない

1 cell に絞れば SPSA は確かに $K = 0$ レベルまで持っていける (C2)。multi-cell で頭打ちになるのは、 $\beta \in \mathbb{R}^{10}$ という設計が cell の多様性に対して表現力不足だから。

解決の方向: β を固定数ではなく、cell ごとに変える仕組みにする。たとえば「直近の innovation 統計を見て、そのときの β を吐く小さな network」= context-conditioned β network。これは Project 1.5 の自然な目標。

6.4.4. 次のモデル形(将来研究)

```
beta_network(  
  直近の field ごと innovation 統計  
  (平均, 分散, 自己相関)  
) -> beta in R^10
```

(sigma, d) を補助入力にすることもできるが、生体はノイズや遅延の数値を直接観測しないので、innovation の時系列だけを主入力にするのが生物学的整合性が高い。

6.5. C4 — ルールはタスク次第: 転移は必ずしも成功しない

6.5.1. どんな実験か

これまでは ReachFixed(リーチ目標が毎エピソード同じ位置)で学習・評価していた。ここでは ReachRandom(目標位置が毎エピソードランダムに変わる)で同じ observer を試す。比較するのは:

1. $K = 0$ baseline
2. $K = 1$ baseline
3. default reliability($\beta_0 = 0, \beta_1 = 0.5$)
4. ReachFixed で学んだ β をそのまま使う(転移)

cell	$K = 0$	$K = 1$	default	転移 β	K_a^* (本来の最適 K)
none, d=0	0.670	0.770	0.781	0.768	$K = 0$
none, d=18	0.642	0.647	0.631	0.680	$K = 0.25$
high, d=0	0.670	0.770	0.776	0.768	$K = 0$
high, d=18	0.642	0.639	0.608	0.692	$K = 0.25$
xhigh, d=0	0.670	0.762	0.771	0.759	$K = 0$
xhigh, d=18	0.642	0.627	0.649	0.686	$K = 1$

(default 列の太字: $K = 0$ を上回った、または同等のセル。)

6.5.2. 何が起きているか — 反転現象

1. ReachRandom では最適 K が cell ごとに違う: $\{0, 0.25, 1\}$ の 3 値が出る (multimodal な構造)。ReachFixed では全 cell で $K = 0$ 一様だったのと対照的。
2. default reliability(学習していない素のルール)は delay 18 の 3 cell のうち 2 つで $K = 0$ より 1~3 cm 良い結果を出す。これは default の $K_f \approx 0.6 \sim 0.7$ が偶然 multimodal な最適 $K = 0.25 - 1$ の中間に位置しているため。
3. 一方、ReachFixed で学んだ β は全 delay-18 cell で $K = 0$ より ~4~5 cm 悪い結果を出す。ReachFixed の学習は「gain を全体的に下げる」方向に進んでいたが、ReachRandom ではその方向が間違っていた。

6.5.3. なぜ転移が失敗するか

ReachFixed の SPSA は「すべての cell で $K = 0$ が正解」という前提のもとで、 β を「gain を低く保つ」方向に最適化した。ReachRandom はそもそも前提が違う (cell ごとに違う最適 K を要求する) ので、同じ β では逆効果になる。

C4 の核心 — 正解のルールはタスク次第

correction-gain ルールの「正解」は、そのタスクで「最適 K がどう分布するか」によって決まる。

- ・「最適 K がどの cell でも同じ」タスクでは、outcome 学習(PSA)が default ルールより良い。
- ・「最適 K が cell ごとに違う」タスクでは、default ルールの方が偶然うまくフィットすることがある。
- ・1 つの β を別のタスクに移植しても改善する保証はない。ルール自体が cell-aware(状況依存)である必要がある。

仮説 (testable): 「最適 K が一様」なタスクで学習した agent は、「最適 K がバラつく」タスクに転移したとき性能が落ちる。これを実験的に確かめられる。

7. 研究者としての作業手順

7.1. 1 日の作業サイクル

1. 研究問いを1文で書く
2. config を1つだけ変える
3. 実行前に expected outcome と fail condition を書く
4. uv run で実行
5. metrics.csv / summary.json を確認
6. 図表を再生成
7. Logs または exchange に判断を書く
8. commit する

7.2. 実験前チェックリスト

演習 / 作業

- ・ ☐ env id は意図通りか (myoArmReachFixed-v0 / myoArmReachRandom-v0)
- ・ ☐ target は paired か、direct-write されているか
- ・ ☐ downstream policy は true state を見ていないか
- ・ ☐ policy 名は joint-space feedback controller / endpoint-error sensorimotor policy / behaviour-cloned policy のどれかに整理されているか
- ・ ☐ action layer (excitation, api_action, activation) が混ざっていないか
- ・ ☐ delay semantics は過去ノートと一致しているか
- ・ ☐ run directory に resolved config が保存されるか
- ・ ☐ random seed / target index / model id が metrics に残るか

7.3. 結果解釈チェックリスト

演習 / 作業

- ・ ☐ state estimation error と task error を混同していないか
- ・ ☐ open-loop oracle と closed-loop oracle を分けているか
- ・ ☐ paired comparison になっているか
- ・ ☐ single-cell の大きな差を一般化しすぎているか
- ・ ☐ downstream policy の state coupling を評価しているか
- ・ ☐ observer sensitivity を reaching performance と混同していないか
- ・ ☐ negative result を捨てずに設計上の知見として記録しているか

8. よくある失敗と対処

8.1. target が paired でない

myoArmReachRandom-v0 では、単に `env.reset(seed=k)` するだけでは target 再現性が保証されない場合がある。Project 1 では target set を自前で持ち、target site を direct-write する。

```
env.reset()
_inject_target(env, target_set.target_pos[episode_index])
mujoco.mj_forward(model, data)
```

paired target が崩れると、estimator 差ではなく target distribution 差を測ってしまう。

8.2. K と H の混同

correction gain の K と rollout supervision horizon の H を混同しない。

K: sensory prediction-error correction gain, $0 \leq K \leq 1$

H: rollout supervision horizon, H in {1, 4, 8}

d: observation delay steps

8.3. behaviour-cloned policy の open-loop 化(Appendix C 教材)

BC は reaching を改善するが、小規模 demo では平均 trajectory を再生するだけになりやすい。この場合、observer output を入力に入れていても、実質的には state-coupled ではない。Project 1 の main result では joint-PD controller を採用し、BC は Appendix C の oracle-supervised diagnostic に降格してある(observer signal の wash-out を示す methodological caution として残す)。

対処:

- DAgger / closed-loop data aggregation
- target-conditioned policy の明示
- state masking の効果検証
- observer sensitivity metric の導入

8.4. endpoint-error policy を optimal controller と呼ぶ

endpoint-error sensorimotor policy は、estimated tip-to-target error を muscle excitation に写像する task-level policy である。LQR、OFC、Riccati equation は解いていない。Project 1 main では joint-PD のみを採用し、endpoint-error policy は Appendix C(oracle-supervised diagnostic)で「joint-PD 以外の state-coupled policy でも同じ qualitative finding が出ること」の確認用として残してある。

対処:

- endpoint-error sensorimotor policy または endpoint-error policy と呼ぶ
- OFC-like, LQR-like, optimal feedback controller と書かない
- absolute reaching と state coupling を分けて評価する
- $K=0$ / $K=1$ / learned で trajectory と action が分かれるかを見る

8.5. SPSA outcome を training-time と deployed eval で混同する

SPSA の per-iter outcome は同じ β で s 個 episode 平均しているが、これは training-time の noisy estimate である。fresh deployment では別 seed の独立 episode で測り直す必要がある。

training-time outcome (smoothed last-20-iter mean) vs
deployed eval (final beta, fresh 10 episodes)

Project 1 の C2 では両者を分けて報告している(0.49 m smoothed training vs 0.56 m deployed)。performance claim は deployed eval が headline、smoothed training は convergence evidence。

8.6. oracle なしを “no oracle access” と書く

abstract / intro で “no oracle access” / “without an offline oracle” と書くと、reviewer に “実用システムは当然 oracle 使えないので trivial” と読まれる。実際の non-trivial claim は training 時に per-condition K-sweep oracle ラベルを要求しない こと。

対処:

- “without per-condition oracle labels at training time”
- “rather than oracle-supervised gain learning”
- Appendix C への明示ポイントを併記

9. Project 2 / Project 3 への接続

9.1. Project 1.5: context-conditioned β network

C3 の structural ceiling は、natural な follow-up project として「context-conditioned β 学習器」を要請する。

beta_network:

input = sliding window of field-wise innovation statistics
(mean, variance, autocorrelation optionally (sigma, d)
as ablation-only auxiliary)
output = beta = {beta_0_f, beta_1_f} in $\mathbb{R}^{10}$

学習 signal は引き続き per-episode reaching outcome を SPSA / REINFORCE / ES のような black-box optimization で用いる(target K のラベルは依然として使わない)。

設計上の question:

- innovation window 長: episode 全体 vs 直近 50-100 step
- context encoder: small MLP / GRU / Transformer
- network output: β そのもの / β への residual / per-step K_f 直接
- ReachFixed train $\rightarrow$ ReachRandom test の transfer benchmark を pre-commit

これは Project 1 の論文 main scope の外だが、C3 の限界を直接解消する次の研究軸として明示。

9.2. Project 2: Bayesian integration

Project 1 では observation noise を field ごとの Gaussian sigma として扱い、within-trial reliability は innovation 二乗から経験的に推定した。Project 2 では、視覚・固有受容・prediction・prior を別情報源として扱う Bayesian framework に拡張する。

Project 1 との橋渡し:

- Project 1 の reliability $r_{f(t)} = \frac{1}{\varepsilon + v_{f(t)}}$ は noise covariance estimate $\hat{\sigma}_f^2(t) = v_{f(t)}$ の inverse として読める
- Bayesian integration では $w_i \propto \frac{1}{\sigma_i^2}$ で weighting $\rightarrow$ Project 1 logistic を $\sigma(\beta_0 + \beta_1 \log r)$ と書いた $\beta_1 = 1, \beta_0 = 0$ の special case として読める
- Project 2 は modality conflict (visual vs proprioceptive)、prior の bias、posterior の uncertainty を陽に扱う

候補実験:

- visual noise と proprioceptive noise を別々に増やす(現 paper は同一 σ vector で field-wise scale)
- sensory conflict(visual と proprioceptive を意図的に矛盾させる)
- target prior を偏らせる(ReachRandom の target 分布操作)
- endpoint bias / variability を測る
- 各 source の online estimated precision を beta_network の input に追加

9.3. Project 3: cortico-cerebellar loop

Project 1 の residual MLP は Wolpert/Kawato box-level の forward model 近似としては妥当だが、小脳 microcircuit そのものではない。Project 3 では、forward model を以下のような module に分ける。

cortical command

$\rightarrow$ pontine-like encoder
 $\rightarrow$ cerebellar-like forward model
 $\rightarrow$ thalamic-like relay
 $\rightarrow$ cortical estimator/controller

LTC / CfC は、連続時間・recurrent・入力依存時定数という点で、Project 3 の候補 forward model として自然である。Project 1 の two-layer adaptation(within-trial reliability + across-trial outcome)は、Project 3 の microcircuit module 間でも同じ構造的役割を果たすと想定: cerebellar forward model のシナプス可塑性 = within-trial dynamics、prefrontal / striatal level の outcome-based meta-learning = across-trial。

落とし穴

TNNLS 投稿前に CfC/LTC PoC を始めると、Project 1 の論文主張がぶれやすい。まず Project 1 の投稿を完了し、その後に別ブランチ / 別 phase として Project 1.5(context-conditioned β) → Project 2(Bayesian) → Project 3(cortico-cerebellar)の順に進める。

10. 演習

10.1. 演習 1: 状態 schema を確認する

```
uv run python -c "from myoarm_fse.envs.factory import make_env from myoarm_fse.envs.extractors import extract_state
env=make_env('myoArmReachFixed-v0') env.reset() s=extract_state(env) print(s.flatten().shape)"
```

期待:

(83,)

10.2. 演習 2: test を通す

```
uv run pytest
```

```
uv run pytest -m myosuite
```

default tests と MyoSuite smoke tests の両方を通す。

10.3. 演習 3: 図表を読み解く

F_reliability_default / F_spsa_single / F_spsa_fullgrid を開き、以下を説明せよ。

- F_reliability_default で default reliability が $K=0$ を 23-29 cm 下回る構造的理由は何か(forward model 強度と reliability の関係)
- F_spsa_single の running mean が $K=0$ baseline 0.50 m に到達するのに、deployed eval は 0.56 m に留まる理由は何か(SPSA noise vs eval seed の関係)
- F_spsa_single の bottom panel で qpos の β_0 が一番低く、 β_1 が一番高い意味を解釈せよ(field-wise specialisation)
- F_spsa_fullgrid で multi-cell SPSA が $K=0$ に届かない理由を構造的に述べよ(single global β の表現限界)
- Appendix C(oracle-supervised)の K_{ol}^* と K_{cl}^* が乖離する条件は何か(forward model multi-step supervision との関係)

10.4. 演習 4: 新しい transfer test を設計する

ReachRandom transfer 以外の transfer 軸(controller 変更、forward model H 変更、noise level 拡張、新しい task variant など)を行う場合、実行前に次を文章で固定せよ。

- 何を replication target とするか
- 何を replication target から外すか
- pass / partial / fail の基準
- target pairing の保証方法
- K_{cl}^* structure(uniform / multimodal)の事前予測
- default reliability vs SPSA-trained β vs $K=0$ の比較順序
- Appendix に入れる最小図表

10.5. 演習 5: context-conditioned β の prototype を設計する

C3 ceiling を超える next-step model を設計せよ。最低限決めるべきこと:

- β network の入力(sliding window 長、何の statistics か)

- ・ β network の出力構造(β 直接 / residual / per-step K_f)
- ・ 学習 signal(per-episode outcome、step-wise reward など)
- ・ 最適化手法(PSA / REINFORCE / ES / 微分可能 surrogate)
- ・ ReachFixed $\rightarrow$ ReachRandom transfer の評価設定
- ・ failure mode(over-fit、変動 high など)の事前予想

実装まで進めなくてよい。設計を 1-2 ページにまとめることが演習。

11. 参考資料

11.1. Repository 内

- ・ README.md
- ・ docs/02_InitialImplementationPlan.md
- ・ docs/03_PaperOutline.md
- ・ docs/03_Project1_Foundations.typ
- ・ paper/main.tex
- ・ figures/F1_system_overview.{pdf,png}
- ・ figures/F_reliability_default.{pdf,png}
- ・ figures/F_spsa_single.{pdf,png}
- ・ figures/F_spsa_fullgrid.{pdf,png}
- ・ figures/F2-F7.{pdf,png}(Appendix C oracle-supervised 教材用)
- ・ scripts/make_reframe_figures.py(C1-C3 figure 再生成)
- ・ scripts/diagnose_reliability_observer.py(per-cell K_f dump)
- ・ scripts/train_reliability_adaptive_v2.py(PSA outer loop)
- ・ src/myoarm_fse/estimators/reliability_adaptive.py

11.2. Obsidian 内

- ・ 20_research/myoarm-fse/_index.md
- ・ 20_research/myoArm新規研究プロジェクト候補_2026-05-09/00_全体研究計画.md
- ・ 20_research/myoArm新規研究プロジェクト候補_2026-05-09/01_Project1_ForwardStateEstimation研究計画.md
- ・ Learn/NeuroControl/小脳forward-modelの数学的実装-residual-MLPとLTC.md

11.3. 文献

- ・ Bernstein, N. A. (1967). The Co-ordination and Regulation of Movements.
- ・ Wolpert, D. M., & Ghahramani, Z. (2000). Computational principles of movement neuroscience.
- ・ Kalman, R. E. (1960). A new approach to linear filtering and prediction problems.
- ・ Mehra, R. K. (1970). On the identification of variances and adaptive Kalman filtering(adaptive Kalman = innovation-based covariance estimation の古典).
- ・ Rauch, H. E., Tung, F., & Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems.
- ・ Spall, J. C. (1992). Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation(PSA の原典).
- ・ Caggiano et al. (2022). MyoSuite.
- ・ Hasani et al. (2021). Liquid Time-constant Networks.
- ・ Hasani et al. (2022). Closed-form Continuous-time Neural Networks.

12. 最後に

この研究を遂行するうえで最も大切なのは、実験を増やすことではなく、次の 4 つを分け続けることである。

4 つの分離

1. state estimation error と closed-loop task error を分ける。

2. agent-available signals(innovation history, per-episode outcome)と non-agent-available oracle レベル(per-condition K-sweep の K^*)を分ける。
3. within-trial layer(per-step reliability)と across-trial layer(episode outcome ベースの β 更新)を分ける。
4. SPSA loop convergence(C2 で示した)と parameterisation 表現能力(C3 が拘束されている)を分ける。

この 4 つを分けて考えれば、Project 1 の実装結果は単なる controller engineering ではなく、forward prediction、innovation-based online reliability、outcome-based across-trial learning の三層構造を持つ生物学的にもっともらしい motor control framework の研究基盤になる。次の自然な拡張は、C3 の structural ceiling を超える context-conditioned β network(Project 1.5)と、それを Bayesian framework に embed する Project 2 である。